UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Traitement et acquisition de données

Rapport de stage ST20 - A2022

INZOUDDINE Monia

Département Énergie
Pas de filière

Entreprise Emasolar
25 rue Saint Jean de Dieu, Bâtiment D
69 007 Lyon
www.emasolar.com

Tuteur en entreprise :
CASSIN Lionel

Suiveur UTBM :
PAIRE Damien

Introduction

L'effet photovoltaïque a été découvert en **1839 par Becquerel**. Certains types de matériaux comme le silicium permettent de générer de l'énergie électrique lorsqu'il entre en contact avec l'énergie solaire.

Aujourd'hui, **8 à 22 %** de l'énergie solaire récoltée par les panneaux photovoltaïques sont **convertie en énergie électrique**. L'objectif en France est d'atteindre la neutralité carbone d'ici 2050, c'est pourquoi, on assiste à un d'enveloppement massif de l'utilisation des énergies renouvelables notamment dans les domaines du transport, de l'industrie du chauffage, de l'agriculture. L'emploi de plus en plus massif des panneaux photovoltaïques nécessite leur **maintenance** afin d'assurer **la longévité** des panneaux, leur **efficacité** et la **sécurité** de leur utilisation.

Pour assurer le bon fonctionnement des panneaux solaires, on utilise de plus en plus l'**IOT « Internet of Thing** » ou internet des objets. L'Internet des objets (IoT) décrit le réseau de terminaux physiques qui intègrent des capteurs, des softwares et d'autres technologies en vue de se connecter à d'autres terminaux et systèmes sur Internet et d'échanger des données avec eux.

Dans notre application, ils nous permettront de couvrir une plus large zone **de maintenance à distance**.

Le projet de stage consiste à permettre à l'entreprise d'utiliser un appareil capable de traiter différentes données d'une installation photovoltaïque comme la puissance, la tension alternative, le courant alternative, ou encore la fréquence. Cela permet d'implémenter une **surveillance** à **distance** du fonctionnement des installations photovoltaïques. Elle a pour but de gagner en **rapidité**, en **clarté** et d'avoir un retour **fiable** des **données de l'installation**. C'est ce qu'on appelle le **monitoring**. Le **datalogger** est un appareil qui collecte les données de l'installation photovoltaïque.

Dans ce rapport, nous détaillerons comment plusieurs solutions techniques ont pu être testé. Elle se compose d'une partie **hardware** et une partie **software.** Pour cela, nous étudierons quel type de **communication,** il existe entre un **onduleur** et un **panneau photovoltaïque**. Nous présenterons ce qui a été étudié jusqu'à présent dans l'entreprise. Nous expliquerons l'architecture du code utilisé le fonctionnement de la partie software. De plus, nous verrons comment la recherche de la partie hardware a été mené afin d'avoir un appareil le plus compatible possible avec le projet. Et enfin nous verrons les différentes **améliorations** mise en place.

Remerciements

Table des matières

Introduction 2

Remerciements 3

Table des illustrations 5

1) Présentation de l'entreprise Emasolar 7

i. Le Groupe Emasolar 7

ii. Le Groupe Emasolar au niveau international, européen et national 8

2) Analyse du sujet 11

iii. Le But du Stage 12

iv. Planning prévisionnel 12

v. Analyse de l'existant 13

a. Partie hardware (électronique) de l'existant 13

b. Partie Software : Partie traitement des données sur logiciel 15

3) Recherche de la partie Hardware 21

4) Le routeur WL-R220LF-d-RS485-EU 22

5) Le routeur Milesight UR32 29

1. Les caractéristiques du routeur Milesight UR32 29

2. La prise en main du routeur 29

3. Les tests réalisés 31

6) Améliorations 38

7) Mon expérience de stagiaire tout au long du semestre 39

8) Conclusion 40

Bibliographie 41

Annexes 42

Table des illustrations

Figure 1 Organigramme du siège social de Emasolar 7
Figure 2 Emplacement des agences EMASOLAR 8
Figure 3 Monitoring Courbe représentant une production journalière idéale 9
Figure 4 Monitoring Courbe représentant une production avec un défaut d'isolement 10
Figure 5 Centrale photovoltaïque Type 11
Figure 6 Présentation du datalogger utilisée WEBDYN SUN par l'entreprise emasolar 11
Figure 7 Planning prévisionnel du stage 12
Figure 8 photo de la rasberry pi 4B+ 13
Figure 9 Photo de la rasberry pi 3B+ 14
Figure 10 tableau comparatif de la rasberry pi 3b+ et la rasberry pi 4B 14
Figure 11 RS485 CAN HAT 15
Figure 12 Rasberry Pi avec RS485 CAN HAT 15
Figure 13 module Sim utiliser face avant (À GAUCHE) face arrière (À DROITE) 15
Figure 14 Logiciel de code VScode 16
Figure 15 Logiciel de code avec installation d'une extension de python 16
Figure 16 première partie initialisation de la récupération des données 17
Figure 17 Deuxième partie envoie des données dans un fichier CSV et vérification des données enregistrées 17
Figure 18 : fichier de configuration INI_Time.ini 18
Figure 19 Fichier de configuration INI_INV.INI, d'une centrale photovoltaïque et de ces onduleurs 18
Figure 20 : Fichier de configuration « INI_config.ini » pour le serveur FTP 19
Figure 21 : Commande dans le crontab 19
Figure 22 Salon du SIDO 21
Figure 23 Routeur WLINK R220 22
Figure 24 Paramètres de l'ordianteur avant la premiière connexion 23
Figure 25 Connexion sur l'interface Openwrt 23
Figure 26 statuts du système du routeur WLINK R220 24
Figure 27 Configuration des données 3G/4G 24
Figure 28 Machine virtuel système ubuntu 25
Figure 29 Archive du routeur 25
Figure 30 Langage informatique Python et PHP trouvée 26
Figure 31 exemple de différents packages 26
Figure 32 accès en SSH au routeur pour transférer les packages et le code 27
Figure 33 taille du code sur le routeur 27
Figure 34 Exemple des dépendances à installer 27
Figure 35 Organisation de la mémoire du routeur 28
Figure 36 Taille d'un exécutable 28
Figure 37 Image du routeur Milesight UR32 29
Figure 38 Réglages sur l'ordinateur pour le routeur Milesight 30

Figure 39 Mise en place de la carte sim sur le routeur MILESIGHT 30
Figure 40 structure d'un projet imposé par le constructeur Milesight..... 31
Figure 41 Informations complétées dans l'interface cellular..... 33
Figure 42 sms envoyer d'un téléphone portable au routeur réception ok 34
Figure 43 sms envoyer du routeur au téléphone portable réception ok 34
Figure 44 structure du projet Datalogger mis dans le routeur 35
Figure 45 Simulation d'une installation photovoltaïque..... 36
Figure 46 Exécution du projet Datalogger 37
Figure 47 Les fichiers csv du projet 37
Figure 49 Fonctionnement de la carte sim Matooma..... 48

1) Présentation de l'entreprise Emasolar

i. Le Groupe Emasolar

EMASOLAR est une (SASU) Société par actions simplifiée à associé. Elle est **spécialisée** dans la **maintenance** et la **supervision** de tout type d'équipements électriques lié à une **installation photovoltaïque**. Emasolar a été fondé en 2011, elle propose ces services de maintenance photovoltaïque à des **particuliers** et des **professionnels**. Le **siège social** d'EMASOLAR est basé à Lyon dans le 7ème arrondissement, et c'est après la rencontre d'entrepreneurs indépendants de chez EMALEC (maintenance électrique) et SOLAPRO (Installateur et distributeur de panneaux solaires) que va se créer EMASOLAR.

Voici un **organigramme des employés** du siège social EMASOLAR.

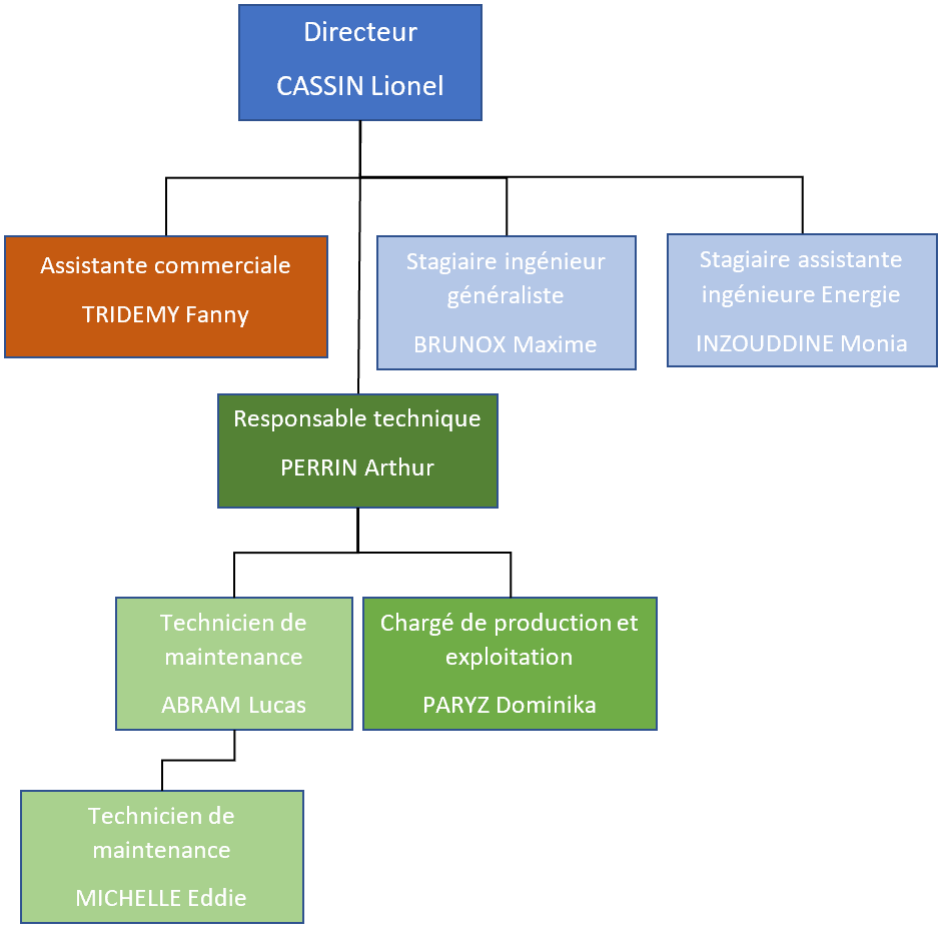


FIGURE 1 ORGANIGRAMME DU SIEGE SOCIAL DE EMASOLAR

ii. Le Groupe Emasolar au niveau international, européen et national

L'entreprise possède **cinq** agences au total partout en France pour une facilité de déplacements liés aux interventions :

- **Lyon** : siège social (69)
- **Montpellier** (34)
- **Nantes** (49)
- **Troyes** (10)
- **Toulouse** (31)

FIGURE 2 EMPLACEMENT DES AGENCES EMASOLAR

C'est à Lyon que s'est déroulé mon stage au sein du **bureau d'étude**. **L'agence de Lyon,** supervise également le travail des techniciens.

Le travail fourni par les techniciens d'EMASOLAR se focalise sur les centrales de panneaux solaires et tout équipement jusqu'au point de livraison (PDL). Les modules concernés sont les **onduleurs**, les **câbles** et leurs **chemins**, les **boitiers** et **armoires électriques** ainsi que le point d'injection chez le client. Deux types de maintenances sont faites chez EMASOLAR, la **maintenance préventive** dites de **prévention**, elle est faite de manière **régulière** et la **maintenance curative** de **réparation d'éventuel problème**.

La société réalise plusieurs types de services, par exemple :

- **Maintenance** des panneaux solaires
- **Dépannage** des centrales photovoltaïques
- **Nettoyage** des panneaux solaires
- **Rénovation** des panneaux solaires
- **Monitoring/télésurveillance** des installations photovoltaïques

Les interventions d'un technicien permettent un **diagnostic complet** de la centrale (endroit où se situe le site des panneaux solaires) ce qui engendre, si besoin, des travaux d'urgences.

Pour la vérification des centrales photovoltaïques, une trame détaillant les étapes à suivre est utilisée par le technicien :

- Vérification de l'état d'entretien du local technique
- Présence d'affichage de sécurité, de documentations et de schéma sur place
- Vérification de la bonne liaison à la terre
- Accessibilité des coffret AC/DC
- Etat des fusibles, des câbles
- Serrage des vis sur les borniers électriques
- Vérification du bon fonctionnement du monitoring (Datalogger)

Comme écrit plus haut, Emasolar effectue également la **télésurveillance**. Celle-ci elle est assurée chaque jour par **l'équipe de monitoring**.

Ce suivi quotidien permet d'assurer de la **bonne production** de la centrale en analysant la **performance** de chacun des onduleurs. Le but du monitoring est de garantir les **revenus solaires.**

Avec ce service, on peut voir qu'Emasolar travail dans la France entière et s'intéresse de près aux dernières technologies notamment à **l'internet des Objets**.

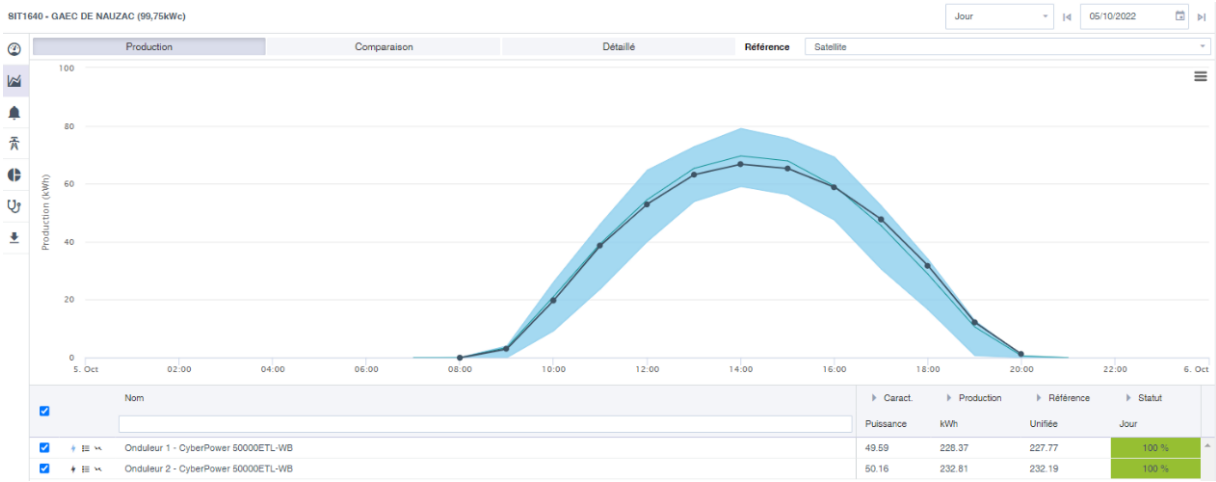


FIGURE 3 MONITORING COURBE REPRESENTANT UNE PRODUCTION JOURNALIERE IDEALE

Sur la **figure 3**, on peut voir une production idéale journalière autour de 50kW. Il y a la courbe bleue qui représente la **production réelle** de la centrale et la **courbe noire** qui

représente une courbe de référence autrement dit **la production théorique**. Les chiffres sont autour de 230 KWh. Comme la production suit la référence on estime le statut à 100%.

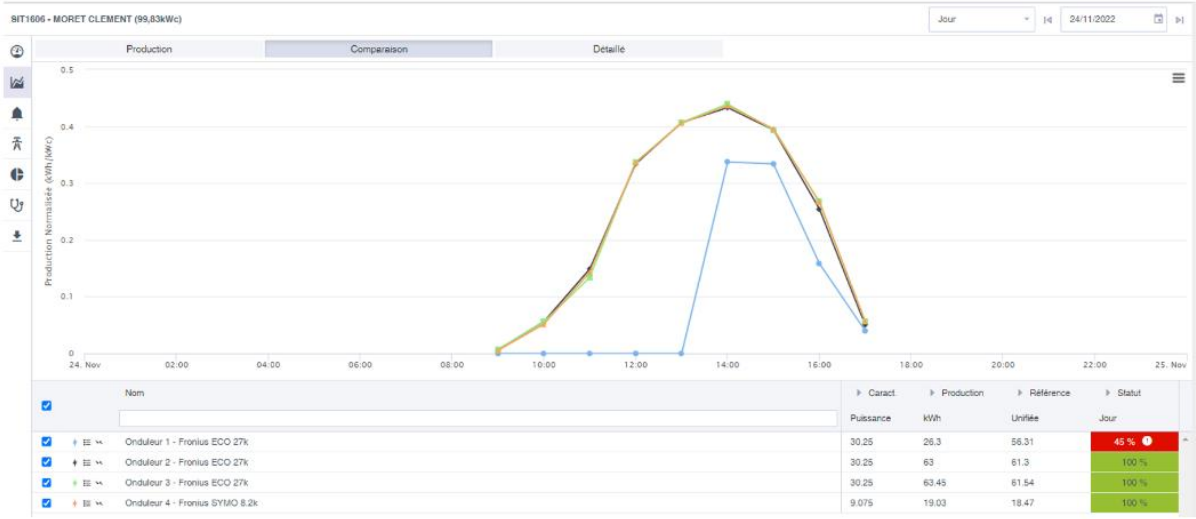


FIGURE 4 MONITORING COURBE REPRESENTANT UNE PRODUCTION AVEC UN DEFAUT D'ISOLEMENT

Sur **la figure 4**, ci-dessus on peut voir quatre onduleurs sur une centrale, on peut voir que le premier onduleur à un **défaut d'isolement** (« le courant fuit »). L'onduleur est à 45% de sa production journalière donc ici pour une référence à 56,31 KWh, la centrale produit 26,3 kWh.

Sur une **installation photovoltaïque,** on retrouve les **panneaux solaires,** les **onduleurs**, et un **datalogger**. Le monitoring nous permet généralement de récolter la Puissance AC, la tension AC, le courant AC par phase, la résistance d'isolement (« le courant fuit »), les variables d'erreur, la température, la fréquence le courant du réseau…etc. Cette liste est non exhaustive mais les principales caractéristiques ont été citées ci-dessus.

Donc, Emasolar a un **<u>rayonnement national important</u>** et grandissant notamment grâce à ces agences dans toute la France.

2) Analyse du sujet

La **télésurveillance** permet de mettre l'accent sur l'utilisation essentielle d'un **datalogger** pour une centrale photovoltaïque. Comme expliquer précédemment, le monitoring permet d'analyser les caractéristiques d'une centrale photovoltaïque.Voici un schéma récapitulatif qui résume le fonctionnement d'une centrale photovoltaïque type.

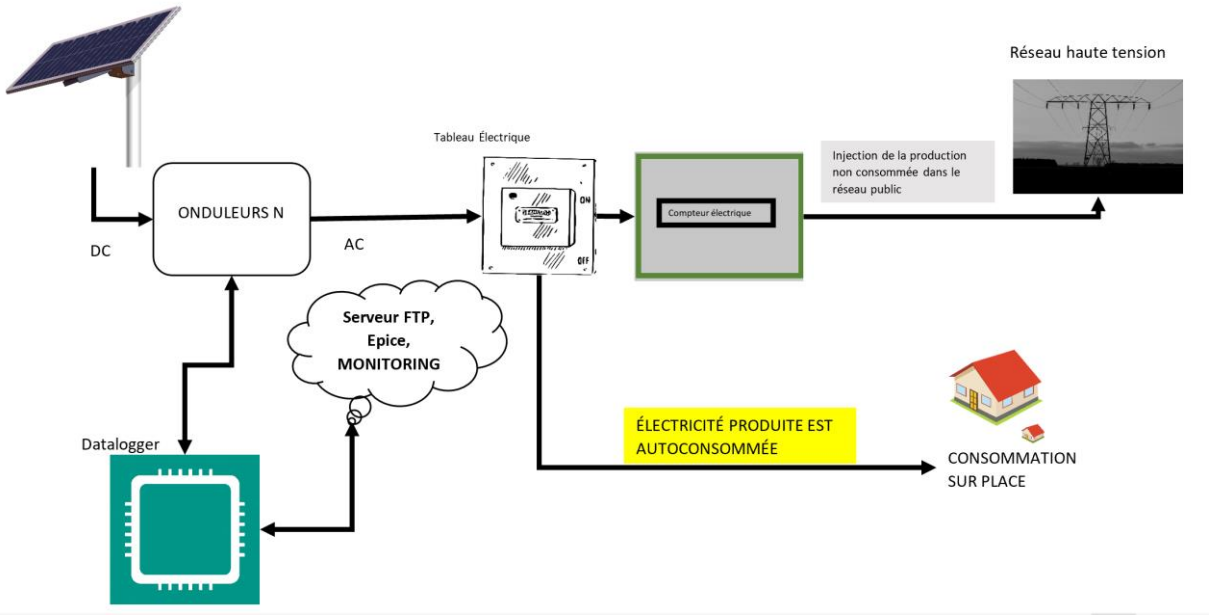


FIGURE 5 CENTRALE PHOTOVOLTAÏQUE TYPE

En effet, les clients qui souhaitent ajouter un DATALOGGER sur leur centrale pour bénéficier d'une télésurveillance le peuvent. Voici comment on pourrait qualifier un datalogger, c'est un **enregistreur de données** est un dispositif automatique, voire programmable, qui enregistre des valeurs de mesure individuelles et des séries de mesure sur une longue période. Les grandeurs sont automatiquement mesurées, souvent numérisées et enregistrées sur un support. Actuellement, sur les centrales des clients de la société EMASOLAR le **Datalogger WEBDYN Sun** est la plus répandue.

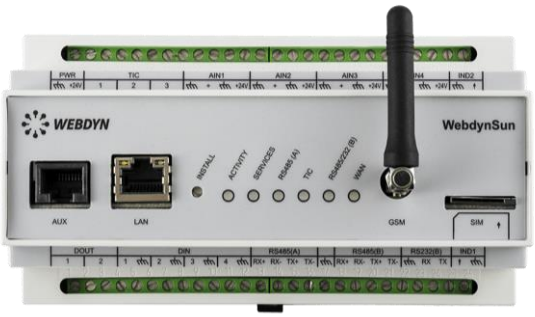


FIGURE 6 PRESENTATION DU DATALOGGER UTILISEE WEBDYN SUN PAR L'ENTREPRISE EMASOLAR

La **Gateway Webdyn Sun collecte les données des centrales solaires** et les envoie sur un **serveur** pour les analyser à distance. Cet appareil rassemble les **indicateurs des onduleurs**, des **compteurs électriques** et des **capteurs environnementaux**.

Il collecte les données des onduleurs par la **liaison RS485.** Ces données sont ensuite envoyées en **format CSV** via des **réseaux mobiles ou Ethernet**.

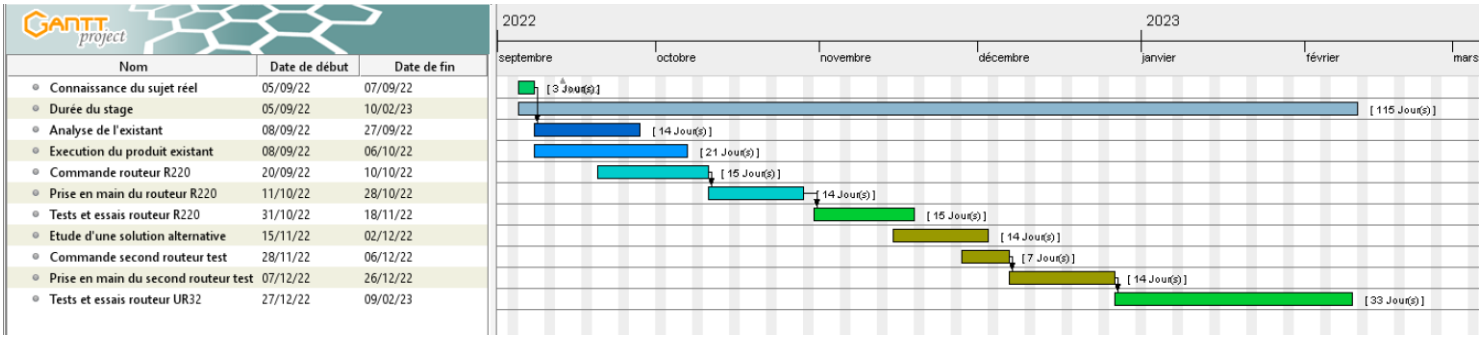
iii. Le But du Stage

Comme énoncé dans l'introduction, le <u>but du stage</u> est que l'entreprise ait un nouvel appareil à moindre cout qui pourrait servir de datalogger. Il aurait un design plus ergonomique et serait facile d'installation.

Après analyse, de la WebdynSun les caractéristiques du datalogger recherchées sont :

- Alimentation
- Entrées diverses (analogiques ou TOR)
- Branchement Ethernet pour une connexion à un réseau
- Branchement antenne GPRS
- Emplacement carte Sim
- Sortie RS485
- Port Wan et Port Lan
- Kit de développement logiciel langage python et C
- Interface de configuration de la WebdynSun

iv. Planning prévisionnel



Nom	Date de début	Date de fin
Connaissance du sujet réel	05/09/22	07/09/22
Durée du stage	05/09/22	10/02/23
Analyse de l'existant	08/09/22	27/09/22
Execution du produit existant	08/09/22	06/10/22
Commande routeur R220	20/09/22	10/10/22
Prise en main du routeur R220	11/10/22	28/10/22
Tests et essais routeur R220	31/10/22	18/11/22
Etude d'une solution alternative	15/11/22	02/12/22
Commande second routeur test	28/11/22	06/12/22
Prise en main du second routeur test	07/12/22	26/12/22
Tests et essais routeur UR32	27/12/22	09/02/23

FIGURE 7 PLANNING PREVISIONNEL DU STAGE

J'ai par la suite réalisé, un planning prévisionnel. Il m'a permis d'avoir une ligne directrice des dates que je devais respecter

v. Analyse de l'existant

Rentrons dans le vif du sujet, après la prise en connaissance du **sujet rée**l, j'ai analysé l'existant.

En effet, un ancien stagiaire, Paul Fleury spécialisé dans la filière électronique et systèmes embarqués a réalisé une analyse approfondie du projet. Il a développé un code écrit avec le **langage informatique python** pour pouvoir récupérer les données des onduleurs, les traiter et les envoyer dans un **fichier CSV s**ur un **serveur FTP**.

J'avais deux grandes parties dans le projet de stage. Il y a la **partie hardware, électronique**. Elle regroupe le support électronique avec lequel les données sont collectées et envoyées. Et il y a la partie **software, informatique**. Dans cette partie, il faut une partie traitement de logiciel pour que les données récoltées par les onduleurs soient correctement lues.

a. Partie hardware (électronique) de l'existant

- La Rasberry PI

Paul Fleury utilisé un **Raspberry Pi 4 Model B et un Rasberry Pi model 3B+**. Le Raspberry Pi 4 est un nano-ordinateur qui se présente sous la forme d'une carte de processeur. Il peut se brancher directement sur un écran ou un moniteur. Son avantage est son faible coût, sa faible consommation d'énergie alimentation externe de 5 V / 3 A. Le Rasberry Pi est un appareil modulable car il a des broches d'entrées de sorties qui lui permettent de connecter d'autres partie électronique. Les appareils Raspberry Pi fonctionnent avec des systèmes d'exploitation libre GNU/Linux, notamment Debian, et des logiciels compatibles. Malgré sa petite carte, son faible coût et son système d'exploitation open source Raspbian, il s'agit toujours d'un ordinateur adapté aux besoins de base en matière de programmation.

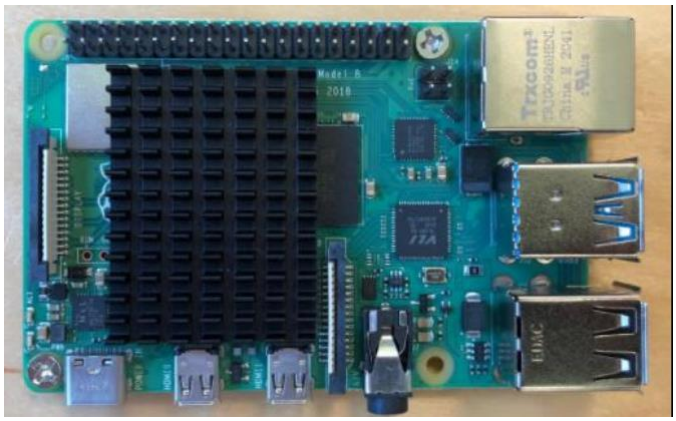


FIGURE 8 PHOTO DE LA RASBERRY PI 4B+

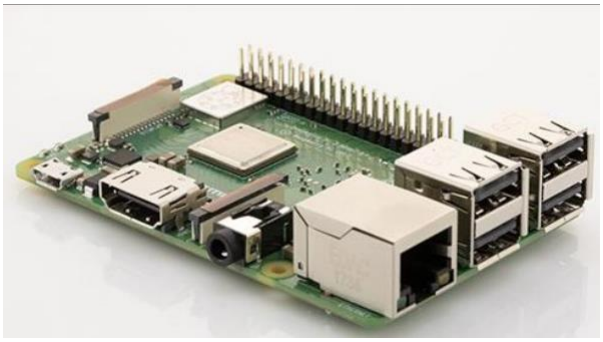


FIGURE 9 PHOTO DE LA RASBERRY PI 3B+

	Rasberry pi 4B (2019)	Rasberry PI model 3B +(2018)
Processeur	Broadcom BCM2711	Broadcom BCM2837B0
Type de noyau	Cortex-A72 (ARM v8) 64-bit	Cortex-A53 64-bit
Nombre de coeurs	4	4
çGPU	VideoCore VI	VideoCore IV
Fréquence de processeur	1,5 GHz	1,4 GHz
USB/Ethernet	2x USB3.0 + 2x USB2.0 + USB-C + Ethernet	4x USB2.0
GPIO	40 pins	40 pins
Power	1,25 A/5V	1,13 A /5V

FIGURE 10 TABLEAU COMPARATIF DE LA RASBERRY PI 3B+ ET LA RASBERRY PI 4B

Dans ce tableau, on peut voir que le Rasberry Pi 4B a un processeur plus puissant que le Rasberry Pi model 3B+. Le Raspberry 4B propose une puce BCM2711 qui permet l'utilisation de 6 UARTs. Emasolar avait donc privilégié l'utilisation de la Rasberry Pi 4B pour le **projet DATALOGGER**.

- La liaison RS485

En plus, de le Rasberry Pi, pour obtenir les données provenant des onduleurs, il fallait un module type RS485, il a donc choisi le module **RS485 CAN HAT**.

FIGURE 11 RS485 CAN HAT

FIGURE 12 RASBERRY PI AVEC RS485 CAN HAT

- Module SIM

Le dernier module nécessaire pour l'envoi des données est un module SIM avec Antenne.

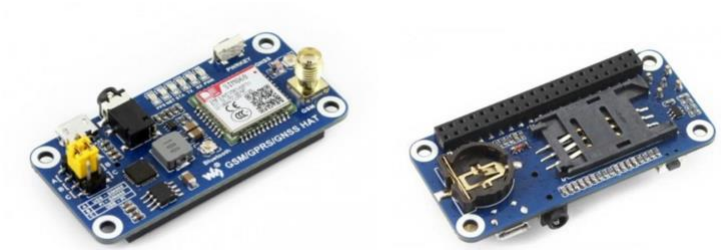


FIGURE 13 MODULE SIM UTILISER FACE AVANT (À GAUCHE) FACE ARRIERE (À DROITE)

Il est essentiel que le module Sim puisse communique avec l'extérieur cependant comme les GPIOS de la Rasberry Pi sont déjà occupée par le **module RS485**, le Hat propose un autre mode de fonctionnement soit en UART soit en USB ici l'USB est donc privilégié pour cela les jumpers sont placés en position A pour une connexion en USB (voir annexe 3).

b. Partie Software : Partie traitement des données sur logiciel

Pour cette partie, la plupart du projet a déjà été codé par l'ancien stagiaire, Paul Fleury en ST50. Il était en Energie filière Electronique et système embarqué.

Pour écrire le code du projet, il a initialement utilisé le **langage Python, des scripts Bash, et des scripts Shells.** Nous détaillerons un peu plus tard à quoi servent ces différents langages. Il a également eu accès au **serveur FTP** de Epice pour envoyer les **fichiers CSVs**. Le but était d'assimiler le code informatique et comme je n'avais jamais étudié le python, il a fallu que j'apprenne assez rapidement le langage. Apprendre le python dans ma carrière en tant qu'**ingénieure** est très important pour moi. Je pense que ce langage permet de réaliser énormément de fonctions d'automatisations des tâches. Ce stage m'a permis aussi de **monter en compétences** dans ce langage. Pour commencer, on utilise le logiciel **VS code** pour ouvrir le projet. On peut installer des extensions qui permettent de lire des fichiers de différents langages. On installe l'extension qui convient pour python, pour les scripts Shell et pour l'extension PHP.

FIGURE 14 LOGICIEL DE CODE VSCODE

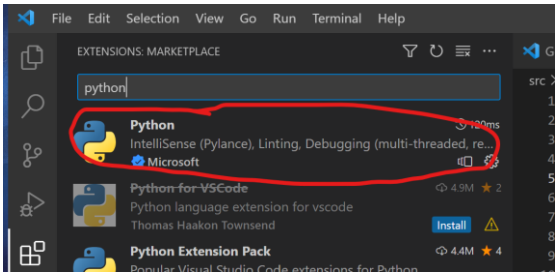


FIGURE 15 LOGICIEL DE CODE AVEC INSTALLATION D'UNE EXTENSION DE PYTHON

En ouvrant le projet, une lecture approfondie du code m'a permis de comprendre le fonctionnement global du projet. Voici qui permet de comprendre la chronologie de l'exécution du projet. Des explications sur chaque scripts et fichiers seront données pour mieux comprendre le projet.

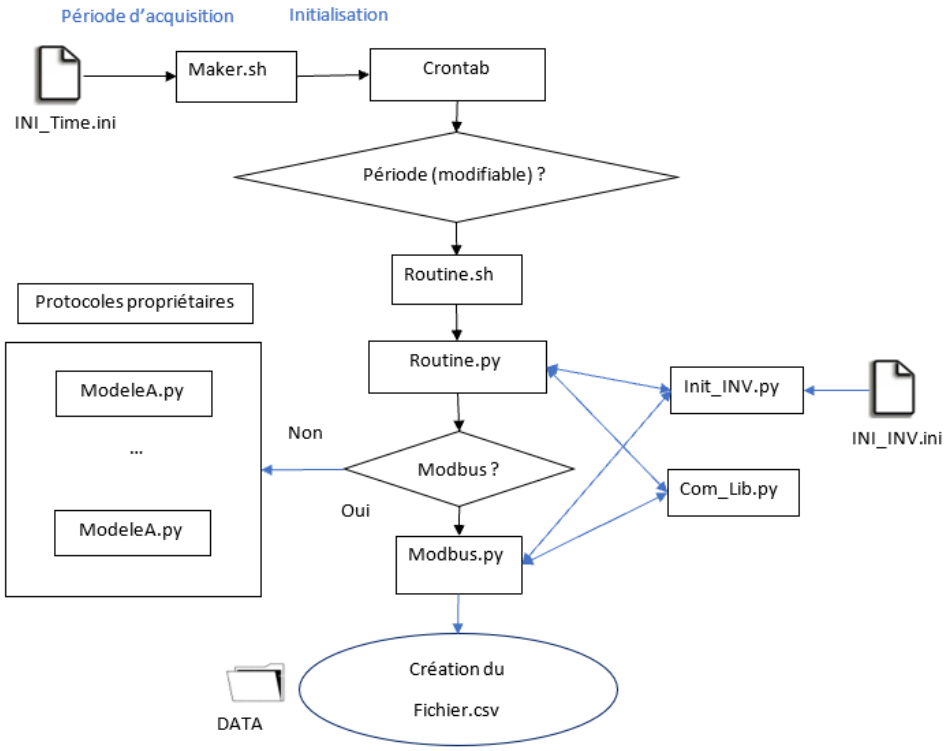


FIGURE 16 PREMIERE PARTIE INITIALISATION DE LA RECUPERATION DES DONNEES

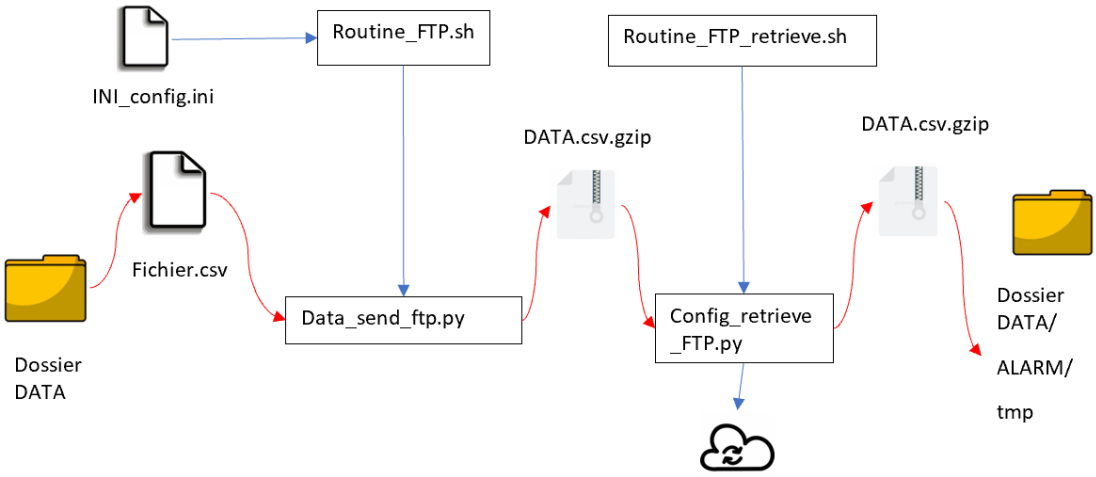


FIGURE 17 DEUXIEME PARTIE ENVOIE DES DONNEES DANS UN FICHIER CSV ET VERIFICATION DES DONNEES ENREGISTREES

Tout d'abord, les fichiers «**. ini** » sont des fichiers de **configurations**, ce sont des fichiers qui contiennent des informations de configuration dans un format simple et prédéfini. Ils se composent de texte brut avec une structure de base composée de propriétés et de sections.

Pour définir le type de communication un premier fichier de configuration, « **INI_Time .ini** ». Il permet de définir la période d'acquisition des données : 10s, le port à travers lequel les données sont transmises, la période d'envoi des données sur le serveur FTP : 59 min. Ensuite, pour la communication Modbus, le mode est mis à 1, la vitesse de transmission est à 9600 bits/seconde, le bits de parité est à 0, la trame est composée de 8 bits sans parité, il y a 1 bits de stop et le temps entre chaque requête en TCP et RTU est de respectivement 2000ms et 500ms. Voir annexe 1 pour plus d'information sur la communication Modbus.

```
[CONFIG-ACQ]
daq_period = 10
PORT = /dev/ttyS0
ftp_period = 59
modbus_mode = 1
modbus_baudrate = 9600
modbus_parity = 0
modbus_databit = 8
modbus_stopbit = 1
modbus_timeout_tcp = 2000
modbus_timeout_rtu = 500
```

FIGURE 18 : FICHIER DE CONFIGURATION INI_TIME.INI

Il y a deux autres fichiers de configuration qui sont définis ainsi.

Sur la **figure 18**, on peut voir « **INI_INV.ini** »

```
[CONFIG-INV]
inv_mode = MB
inv_n = 3
inv_model_0 = HUAWEI
inv_addr_0 = 1
inv_var_0 = HUAWEI_Modbus_v4.txt
inv_model_1 = KACO
inv_addr_1 = 2
inv_var_1 = KACO_Modbus_v1.txt
inv_model_2 = KOSTAL
inv_addr_2 = 3
inv_var_2 = KOSTAL_Modbus_v1.txt
```

FIGURE 19 FICHIER DE CONFIGURATION INI_INV.INI, D'UNE CENTRALE PHOTOVOLTAÏQUE ET DE CES ONDULEURS

Sur la **figure 19**, on a d'abord le mode de communication de la centrale : Modbus, le nombre d'onduleurs de la centrale photovoltaïque : trois, les différents types d'onduleur utilisés HUAWEI, KACO et KOSTAL, leur adresse et les fichiers de définitions qui y sont associés.

Et le dernier fichier, **« INI_config.ini »** de configuration sert à déterminer l'envoi des données csv via une carte sim sur un serveur FTP.

```
[FTP]
gprs_apn = matooma.m2m
gprs_login = TEST
gprs_password = 1234
gprs_phonenumber = 05596
nosim = False
ftp_server = ftpupload.net
ftp_login = epiz_28172809
ftp_password = QVXALg76IBv
ftp_port = 21
ftp_dirconfig = /CONFIG/
ftp_dirdef = /DEF/
ftp_dirdata = /
ftp_dirlog = /LOG/
ftp_diralarm = /
local_edit = 1
```

FIGURE 20 : FICHIER DE CONFIGURATION « INI_CONFIG.INI » POUR LE SERVEUR FTP

Sur la **figure 20**, on a d'abord l'APN, « Access Point Name » constitue l'ensemble des informations de connexion relatives à un réseau donné, en fonction d'un opérateur. Ensuite, on a le login, le mot de passe, le numéro de téléphone, la présence ou non d'une carte sim. Puis, il y a les informations du serveur FTP fournis par l'entreprise, le port du serveur FTP, et les différents emplacements des fichiers voire **figure 17**.

Sur la **figure 16**, on peut voir qu'après avoir défini la période d'acquisition, un script Shell initialise le lancement des différentes tâches. Un script Shell permet depuis le terminal d'un ordinateur d'exécuter une tâche, et un programme informatique.

Le script shell **« Maker.sh »** est un script d'installation et de création des tâches à effectuer et permet la mise en place du Crontab.

```
* 12 * * * bash -c "/home/pi/src/git/RPI/Jobs/Routine_Wipe.sh -d" >> /var/log/out_ftp.txt
```

FIGURE 21 : COMMANDE DANS LE CRONTAB

Sur la **figure 21**, en rouge il y a l'indication du minutage fixé pour lancer la commande en vert. Le minutage fonctionne sur 5 notations à respecter pour chaque tâche à exécuter :

mm hh jj MMM JJJ – Commande

- mm : minutes
- hh : heures
- jj : jour du mois
- MMM : nom du mois
- JJJ : jour de la semaine

La tâche (**Figure 21**) est lancée tous les jours à 12 heures. Cela nous permet donc de planifier le lancement des scripts pythons régulièrement.

Les scripts Shell :

- **Routine_ACQ.sh** : Script de vérification de la présence d'une communication onduleur, si aucune communication n'est en cours, elle démarre.
- **Routine_FTP.sh** : Script de vérification de la présence d'une communication vers un serveur FTP, si aucune communication n'est en cours, elle démarre.
- **Routine_FTP_retrieve.sh** : Script de vérification de la présence d'une communication depuis un serveur FTP, si aucune communication n'est en cours, elle démarre.

Les scripts Pythons

- **Routine.py** : Vérification du protocole de communication utilisé (TCP, RTU ou propriétaire…) et lancement de la communication.
- **Modbus.py** : Script de communication onduleur en Modbus RTU.
- Init_INV.py : Script d'initialisation des variables issues des fichiers de configurations.
- **Com_Lib.py** : Script regroupant des fonctions de traitement de données
- Modele.py : Ici on crée un script pour chaque modèle d'onduleur qui ne communique qu'avec son protocole propriétaire.
- **Data_send_ftp.py** : Script pour configurer l'envoi des données sur un serveur FTP
- **Config_retrieve_FTP.py** : Script qui permet de télécharger les fichiers depuis le serveur FTP

3) Recherche de la partie Hardware

Une fois que le sujet réel a pu être analysé et que j'ai commencé à étudier le code informatique développé. J'ai commencé à rechercher des appareils électroniques qui correspondaient à nos critères.

J'ai donc redéfini les critères principaux suivants :

- Alimentation
- Entrées diverses (analogiques ou TOR)
- Port WAN et LAN
- Branchement antenne GPRS
- Emplacement carte Sim
- Sortie RS485
- Port Wan et Port Lan
- Kit de développement logiciel langage python et C

Au début, nous avions choisi des nano-ordinateurs comme le Raspberry Pi. Puis, après réflexion, nous avons décidé qu'il fallait un appareil électronique avec un design ergonomique et plus professionnel. De plus, nous n'étions pas certains que le Raspberry PI était assez robuste pour avoir une longévité de fonctionnement de plusieurs années en continu. Enfin, certains des clients Emasolar sont des agriculteurs, le Datalogger est donc posée dans des endroits où il n'est pas forcément protégé des intempéries. Il nous fallait un appareil avec un design professionnel, robuste assez compact et petit.

Durant mon stage, j'ai eu la chance d'assister au **salon professionnel du SIDO** qui s'est déroulé à Lyon. **SIDO Lyon** est le salon B2B de la convergence des technologies IoT, IA, XR & Robotique pour la transformation digitale des entreprises.

FIGURE 22 SALON DU SIDO

Dans ce salon, les professionnels de la technologie nous ont orienté vers l'utilisation d'un routeur. Après le salon, j'ai effectué plusieurs recherches sur les routeurs qui correspondaient à nos critères. Avec mon tuteur, nous sommes finalement tombés d'accord sur un routeur qui correspondaient à nos attentes : le routeur **WL-R220LF-d-RS485-EU.**

4) Le routeur WL-R220LF-d-RS485-EU

Les caractéristiques du routeur sont :

- Carte Sim 3G /4G
- Port LAN
- Port RS485
- Connexion VPN
- Connexion SSH
- SPI Nor Flash – 16MB
- NAND Flash 128MB (voir annexe)
- DDR2： 16Bit 64MB (Voir annexe)
- CPU : Atheros AR9344, 533MHz
- OpenWRT Os

FIGURE 23 ROUTEUR WLINK R220

La **mémoire Flash** est un mémoire non volatile, elle n'efface donc pas les données après extinction du routeur.

Le plus important pour le « **projet datalogger »** est que le routeur choisi plus haut fonctionne avec le système d'exploitation OpenWRT, c'est un système d'exploitation qui permet d'exécuter du code informatique avec un système de **package**. Autrement dit, pour pouvoir, exécuter le code écrit en python, je dois d'abord télécharger les packages pythons.

Tout d'abord, j'ai pris en main le routeur pour cela, j'ai réalisé les branchements nécessaires : le câble d'alimentation branchée à la prise secteur, le câble RJ45 relié à l'ordinateur et la carte Sim insérée.

Pour accéder à l'interface du routeur sur l'ordinateur, il faut mettre les bons paramètres dans l'ordinateur comme suit :

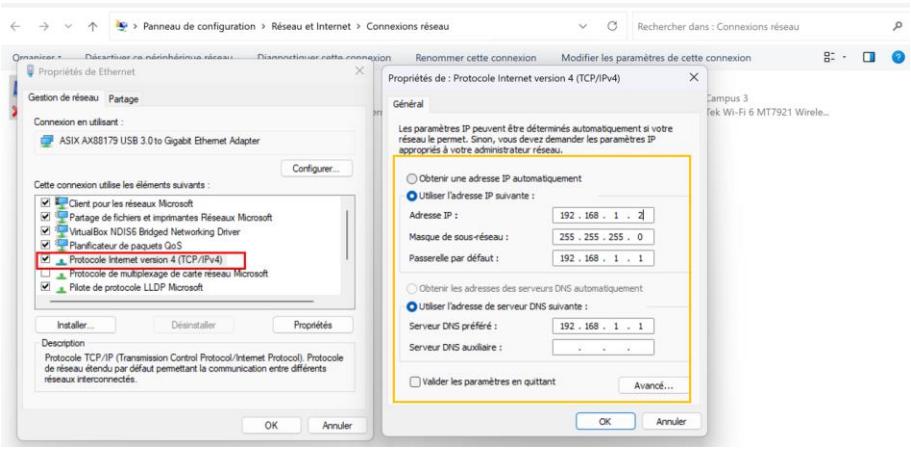


FIGURE 24 PARAMETRES DE L'ORDIANTEUR AVANT LA PREMIIERE CONNEXION

Cela permet d'être dans la bonne plage d'adresse du routeur.

Puis, il faut se connecter à l'interface OpenWrt avec l'adresse Ip donné par le constructeur. Le constructeur donne également le login et mot de passe à la première utilisation.

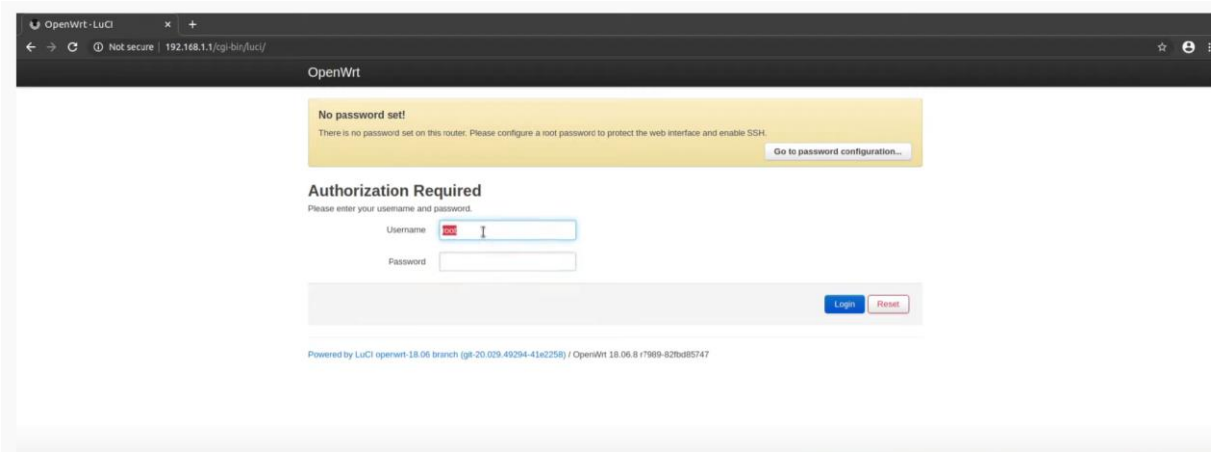


FIGURE 25 CONNEXION SUR L'INTERFACE OPENWRT

Après la première connexion sur le routeur, la première page où se retrouve l'utilisateur est la suivante.

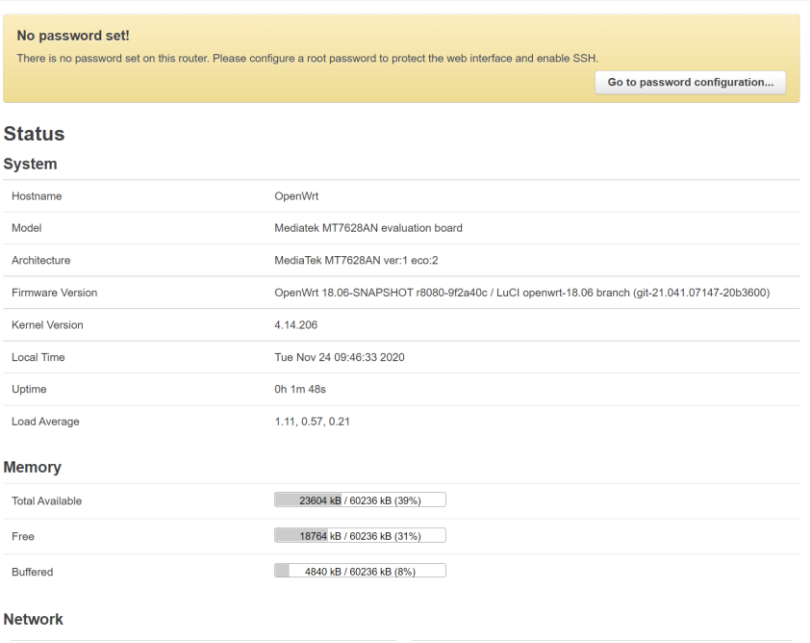


FIGURE 26 STATUTS DU SYSTEME DU ROUTEUR WLINK R220

Elle renseigne l'état du système général du routeur : le système d'exploitation utilisée, le modèle, utilisé la mémoire …etc.

Enfin, j'ai tenté de renseigner les coordonnées pour faire fonctionner le routeur grâce à une connexion 3G/ 4G cependant cela ne fonctionnait pas comme prévu.

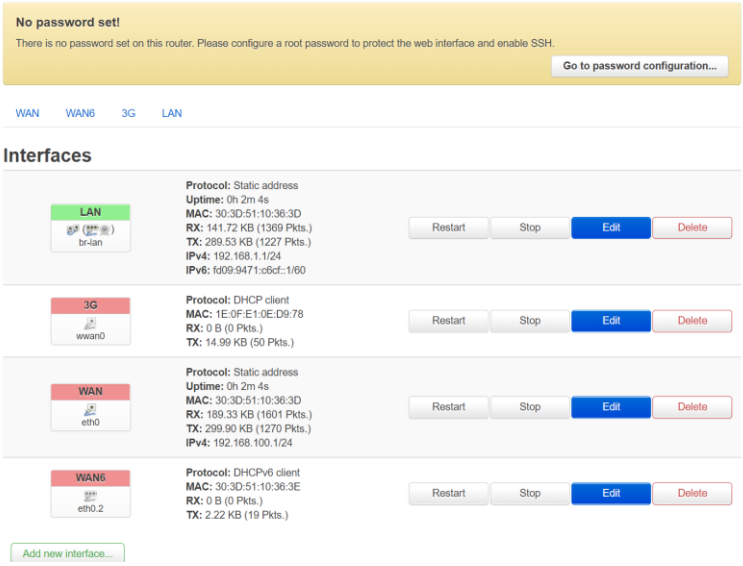


FIGURE 27 CONFIGURATION DES DONNEES 3G/4G

Pour réaliser ces étapes, j'ai eu l'aide du directeur technique Arthur Perrin, il m'a aidé pour les branchements, le paramétrage et la configuration de la carte Sim.

Cependant, tout ne s'est pas passé comme prévu car la notice d'utilisation n'était pas à jour. Les informations ne correspondaient pas à ce qu'on pouvait retrouver sur le routeur.

J'ai donc décidé de passer à la suite du projet c'est-à-dire la mise en place du code python sur le routeur WLINK R220 et de revenir sur la carte Sim un peu plus tard.

La notice était erronée et j'avais très peu d'informations claires sur la démarche que je devais suivre pour télécharger les packages python sur le routeur. C'est pourquoi, il a été très important que j'échange régulièrement avec le support technique la société ES France (vendeur du routeur WLINK R220) pour avoir plus d'informations. Après quelques échanges, j'ai pu établir une liste des étapes que je devais effectuer pour télécharger les packages pythons :

Etape 1 : Je devais travailler sur un environnement linux, pour accéder à l'interface d'OpenWRT, je travaillais sur une interface Windows jusqu'à présent. Donc pour ça, j'ai téléchargé une Virtual Box sur l'ordinateur. Une Vitual Box permet d'installer différents systèmes d'exploitation sur le même ordinateur. J'ai installé le système Ubuntu.

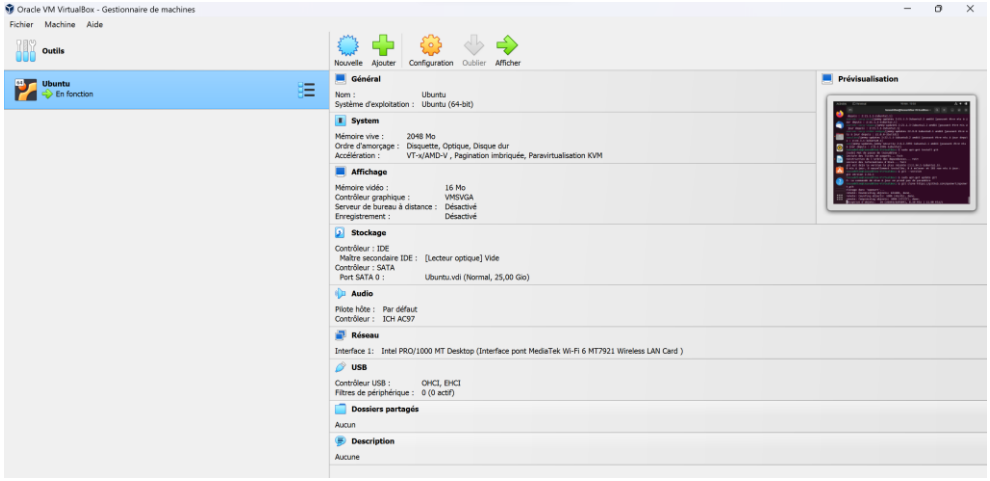


FIGURE 28 MACHINE VIRTUEL SYSTEME UBUNTU

Etape 2 : Je devais télécharger la bonne archive celle qui correspondait au routeur WLINK R220

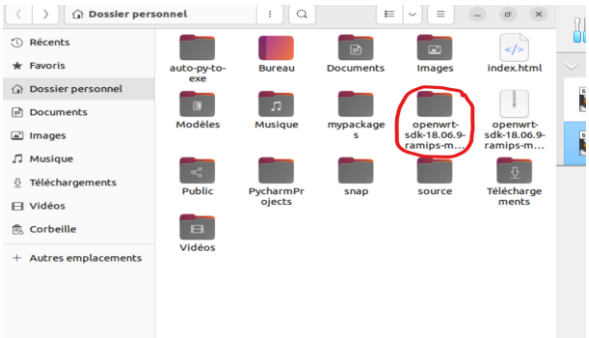


FIGURE 29 ARCHIVE DU ROUTEUR

Etape 3 : Je devais accéder au dossier avec le terminal afin d'installer les « feeds » à jour autrement dit les « flux de donner ». En tapant dans le terminal, « Make menuconfig » je pouvais accéder au menu de configuration du routeur et lancer la configuration des packages python et PHP.

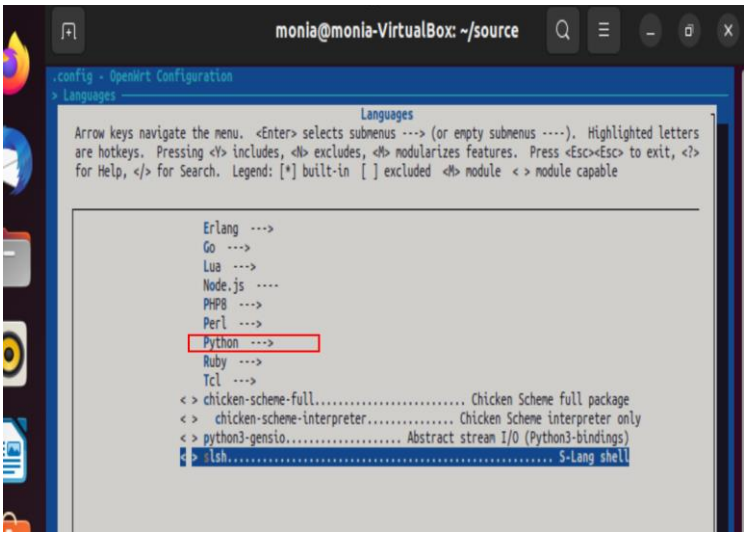


FIGURE 30 LANGAGE INFORMATIQUE PYTHON ET PHP TROUVEE

Etape 4 : Apres la compilation, je trouvais les packages sous format «. Ipk »dans un répertoire précis. Et je devais choisir les packages et les installer dans le routeur avec la commande : opkg install <nom>.ipk.

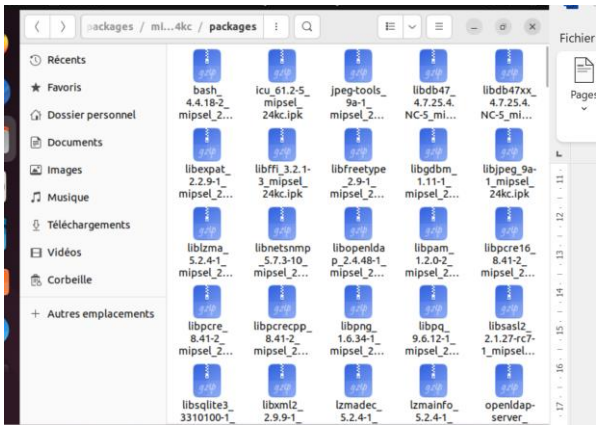


FIGURE 31 EXEMPLE DE DIFFERENTS PACKAGES

Etape 5 : Accéder en SSH au routeur pour pouvoir transférer les packages et le code informatique

```
BusyBox v1.28.4 () built-in shell (ash)

 _______                     ________        __
|       |.-----.-----.-----.|  |  |  |.----.|  |_
|   -   ||  _  |  -__|     ||  |  |  ||   _||   _|
|_______||   __|_____|__|__||________||__|  |____|
         |__| W I R E L E S S   F R E E D O M
-----------------------------------------------------
OpenWrt 18.06-SNAPSHOT, r8080-9f2a40c
-----------------------------------------------------
root@OpenWrt:~# ls
php7-cgi_7.2.34-1_mipsel_24kc.ipk  python3_3.6.12-1_mipsel_24kc.ipk
php7_7.2.34-1_mipsel_24kc.ipk
root@OpenWrt:~# cd /tmp
root@OpenWrt:/tmp# ls
TZ                hosts             luci-modulecache  run               usr
dhcp.leases       lib               opkg-lists        shm
dnsmasq.d         lock              overlay           state
etc               log               resolv.conf       sysinfo
extroot           luci-indexcache   resolv.conf.auto  tmp
root@OpenWrt:/tmp#
```

FIGURE 32 ACCES EN SSH AU ROUTEUR POUR TRANSFERER LES PACKAGES ET LE CODE

J'ai pu accéder au routeur en SSH et commencer à transférer les paquets pythons.

Taille sur le disque :	488 Ko (499 712 octets)

FIGURE 33 TAILLE DU CODE SUR LE ROUTEUR

En ce qui concerne la taille du code elle représentait seulement ½ dixième de la mémoire du routeur. Pour plus de précision, lorsque l'on veut exécuter un code écrit en langage python dans le routeur WLINKR220, on peut installer soit uniquement l'interpréteur python3 soit tous python3 et ces dépendances. Pour notre projet, installer uniquement l'interpréteur aurait été largement insuffisant pour exécuter le code de python montrer au départ. Je devais donc installer python3, ces dépendances, les bibliothèques et le code à exécuter pour tester le projet.

```
hosts                                resolv.conf                          tmp
root@OpenWrt:/tmp# opkg install python3-pip_18.1-2_mipsel_24kc.ipk
Installing python3-pip (18.1-2) to root...
Collected errors:
 * satisfy_dependencies_for: Cannot satisfy the following dependencies for python3-pip:
 *      python3
 *      python3-setuptools
 *      python-pip-conf
 * opkg_install_cmd: Cannot install package python3-pip.
root@OpenWrt:/tmp#
```

FIGURE 34 EXEMPLE DES DEPENDANCES A INSTALLER

Ce qui faisait en tout plus de 10MB, j'ai cherché différentes solutions par exemple installer python3-light une version alléger de python3. Cependant, cette solution n'était pas envisageable, nous avions besoins de tout le système python3. De plus, les bibliothèques de pythons qui manipulent les tableaux multidimensionnels comme Numpy prennent Minimum 5MB de données sans compter les données des onduleurs qui auraient été enregistré dans le routeur.

```
BusyBox v1.28.4 () built-in shell (ash)

  _______                     ________        __
 |       |.-----.-----.-----.|  |  |  |.----.|  |_
 |   -   ||  _  |  -__|     ||  |  |  ||   _||   _|
 |_______||   __|_____|__|__||________||__|  |____|
          |__| W I R E L E S S   F R E E D O M
 -----------------------------------------------------
 OpenWrt 18.06-SNAPSHOT, r8080-9f2a40c
 -----------------------------------------------------
root@OpenWrt:~# df -h
Filesystem                Size      Used Available Use% Mounted on
/dev/root                 5.0M      5.0M         0 100% /rom
tmpfs                    29.4M      2.0M     27.4M   7% /tmp
/dev/mtdblock6            9.4M      4.5M      4.8M  48% /overlay
overlayfs:/overlay        9.4M      4.5M      4.8M  48% /
tmpfs                   512.0K         0    512.0K   0% /dev
root@OpenWrt:~#
```

FIGURE 35 ORGANISATION DE LA MEMOIRE DU ROUTEUR

Comme on peut le voir sur la figure 34 la mémoire tmp contient 29, Mb mais cette mémoire est volatile ce qui signifie que la mémoire s'efface lorsque le routeur s'éteint. En revanche, la mémoire overlay est une mémoire non temporaire les données sont conservés même après extinction du routeur. D'autres solutions ont été prises en compte comme convertir le code en exécutable après plusieurs tests l'exécutable prenait plus de 21Mb. Ce qui n'était pas envisageable.

Routine.exe	13/12/2022 16:35	Application	21 079 Ko

FIGURE 36 TAILLE D'UN EXECUTABLE

Enfin, nous avons pris la décision avec mon tuteur Monsieur Cassin de choisir un autre routeur pour avoir des résultats différents. Le deuxième appareil choisi est le routeur Milesight UR32.

5) Le routeur Milesight UR32

1. Les caractéristiques du routeur Milesight UR32

- Tension d'alimentation 9-48V
- Système d'exploitation OScisco
- CPU ARM Cortex-A7, 528 MHz Memory
- 128 MB DDR3 RAM et 128MB Flash
- Extension avec carte micro SD
- 3 antennes : 1 pour la connexion sim, 1 pour le WIFI/ GPS et 1 pour pour Aux
- 1 carte SIM
- 1 port RS232/RS485
- 1 port LAN/WAN
- 1 port LAN
- SDK Python27
- VPN

FIGURE 37 IMAGE DU ROUTEUR MILESIGHT UR32

2. La prise en main du routeur

Comme avec le routeur précédent, j'ai édité les **bons paramètres** dans l'ordinateur, c'est-à dire que j'ai tapé une adresse **IP statique**. La création d'IP statiques permet d'éviter les conflits de réseau qui pourraient empêcher certains appareils de fonctionner correctement. Une adresse IP statique (également appelée adresse IP manuelle ou configuration IP statique) est une adresse IP qui reste inchangée dans le temps. L'adresse IP reste la même chaque fois que l'on se connecte depuis le même endroit. L'adresse IP peut changer si on se connecte à un autre réseau à un autre endroit. À chaque fois, que je me connecte au routeur je tape la même adresse. Pour cela, on écrit dans les champs à renseigner l'adresse IP compris dans une plage définit par le constructeur du routeur. On renseigne également le masque de sous-réseau auquel on essaie de se connecter, celui-ci doit être le même que celui de l'ordinateur où on effectue les opérations puis on renseigne l'adresse de la passerelle et enfin le serveur nom de domaine ou encore « DNS ».

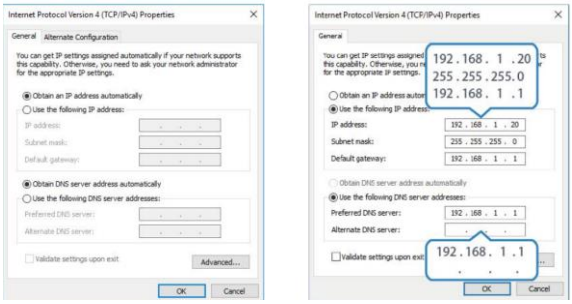


FIGURE 38 REGLAGES SUR L'ORDINATEUR POUR LE ROUTEUR MILESIGHT

Puis, on édite les bons paramètres pour faire fonctionner le routeur en 3G/4G. pour la connexion en 3G/4G comme précédemment, la seule caractéristique à renseigner est le l'**APN**.

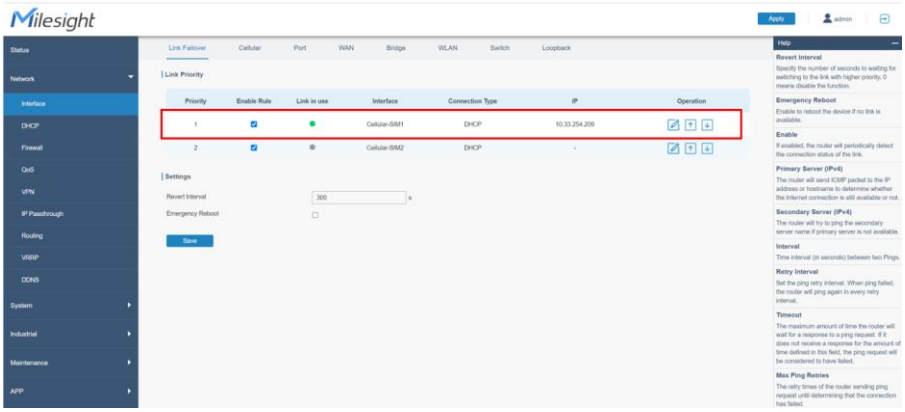


FIGURE 39 MISE EN PLACE DE LA CARTE SIM SUR LE ROUTEUR MILESIGHT

Les réglages effectués pour la carte Sim ont parfaitement fonctionné.

Dans ce routeur, il faut également mettre en place le kit de développement logiciel python ou **SDK python**. Il est fourni par le fournisseur « Sparwan ». Pour cela, il faut au préalable insérer obligatoirement une carte SD et la formater grâce à la plateforme du routeur afin que toutes les données supplémentaires installées soient stockées en mémoire par la carte SD. Par la suite, il a fallu se connecter en SSH au routeur.

Le routeur utilise un système d'exploitation **Cisco IOS**. Cisco Internetworking Operating System (IOS) est un système d'exploitation optimisé pour l'inter réseautage. Cependant, j'étais très limitée avec les commandes du terminal. Je n'avais pas accès en administrateur sur le routeur. Je pouvais seulement effectuer quelques réglages comme télécharger les bibliothèques.

J'ai pu télécharger les différentes **bibliothèques pythons** avec la commande « pip install <bibliothèque> ». Cependant, les bibliothèques qui étaient gourmandes en ressource ou qui nécessitaient le compilateur GCC comme Numpy ne fonctionnaient pas sur le routeur.

La société « Sparwan » (fournisseur du routeur UR32) n'avait eu aussi pas connaissance de cette caractéristique de ce routeur.

Pour tester le kit de développement python sur le routeur, certaines adaptations ont été nécessaires. La structure des fichiers était imposée par le fabricant du routeur.

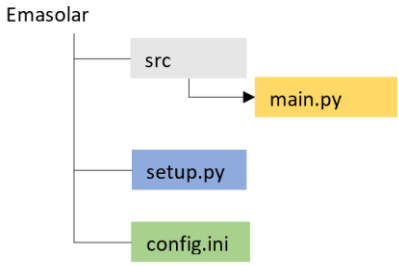


FIGURE 40 STRUCTURE D'UN PROJET IMPOSE PAR LE CONSTRUCTEUR MILESIGHT

La **figure 40**, nous présente la schématisation de la structure des projets implantés dans le routeur. Cette structure est à respecter avec n'importe quel projet.

- « src » contient tous les scripts pythons
- « setup.py » est un module utilisé pour créer et distribuer des packages Python. Il contient généralement des informations sur le package, telles que son nom, sa version et ses dépendances, ainsi que des instructions pour la construction et l'installation du package.
- « config.ini » est le fichier de configuration.
- Et enfin le « main.py » permet d'exécuter le code python.

Pour visualiser la sortie/résultat du code, j'ai eu accès à une plateforme par les fournisseurs du routeur. Ils m'ont fourni le login et mot de passe et j'ai pu avoir une fenêtre de sortie du code. Différentes fonctionnalités comme des « logs » permettent de debugger le programme. Par la suite, avec mon tuteur nous avons donc décidé de modifier le code python initial afin de pouvoir faire les tests avec des onduleurs.

3. Les tests réalisés

Pour le bon déroulement du projet, il y avait plusieurs tests à valider afin de vérifier que les caractéristiques techniques du routeur correspondaient bien au résultat final attendu. Pour cette partie, il a été plus prudent de tester certaines fonctionnalités du routeur indépendamment du projet initial à cause des différents obstacles rencontrés.

Premier test

L'un des premiers tests à valider avec le routeur est **l'envoi du fichier CSV sur un serveur FTP** avec le langage python version 2.7.

Pour le bon fonctionnement des différents codes implantés sur le routeur, j'ai précisé quel type de code on utilise normalement après python3.5 le code type utilisé est « UTF 8 ».

Tout d'abord, il fallait que je me connecte en administrateur au routeur. J'ai donc initialement changé le **firmware** du routeur comme indiquer par le fournisseur. Selon le Datalabcenter, un firmware est un programme informatique intégré dans un matériel, qui participe à son bon fonctionnement et qui lui permet d'évoluer (via l'installation de mises à jour) sans avoir besoin de remplacer ce matériel ou de revoir son design. Dans les routeurs, ce type de fichier a très souvent une extension « .bin ». Autrement dit, le firmware est **l'interface Milesight** sur lequel je me connecte pour interagir avec le routeur et le configurer. Pour accéder à l'interface Milesight en administrateur, le login et le mot de passe change ils sont alors définis par le constructeur. Le type de commande utilisé ici sont **les commandes OpenWrt** certaines commandes linux peuvent être utilisés par exemple « mv », « cp », « ls », « cd »…etc

En administrateur, je peux alors accéder aux projets via le terminal grâce à un **accès en SSH** ce qui n'était pas le cas avant. Je peux aussi lancer le code à partir du terminal avec la commande « python <code.py> ».

Les avantages d'être en administrateur est que le code est plus rapide à l'exécution, on peut aussi voir les fichiers qui sont créés. On peut lister les différents fichiers crées.

Pour cela, j'ai réalisé une architecture de code simple. Je recherche les bibliothèques qui permettent d'envoyer un fichier CSV, il y a **Ftplib, csv et time**.

Le processus consiste à créer et écrire dans du fichier CSV. Puis, on se connecte au serveur FTP fournit par l'entreprise Emasolar, avec le lien du serveur FTP, le login et le mot de passe. Ensuite, on déplace le fichier csv sur le serveur FTP. On déplace le fichier csv sur le serveur FTP grâce à la méthode stobinary(). Cette méthode storbinary() initie un transfert de fichier d'un client FTP vers un serveur FTP à l'aide de la commande FTP STOR. Et enfin, on se déconnecte du serveur.

Si on reprend le code qui crée un fichier csv et qui l'envoie sur un serveur FTP. On peut vérifier que le fichier csv de trouve au-dessus du dossier src. Ce test permet de vérifier la bonne connexion de la carte sim sur le réseau et de vérifier que le fichier se crée et s'envoie sur le serveur FTP. Dans cette partie il est important de souligner que pour le bon fonctionnement du projet je travaillais exclusivement avec le terminal. Il a été réalisé avec **succès**.

Deuxième test

Le second test consiste à envoyer un sms d'un téléphone vers le routeur et confirmer sa réception. Puis d'envoyer un sms du routeur vers le téléphone. Et enfin d'avoir une réponse automatique du routeur après l'envoi d'une certaine commande prédéfini par le fabricant. Ces commandes se nomment commande CLI.

Le but de ce test est de permettre à un technicien maintenance d'envoyer un sms au routeur, et en retour le routeur émet un message automatique s'assurant du bon fonctionnement de la centrale.

Pour cette partie je me suis connectée à l'interface Milesight, pour rentrer les coordonnées de la carte sim. Durant la réalisation de cette partie, on renseigne l'APN défini plus haut. Le code PIN et le numéro de téléphone de la carte sim.

La difficulté rencontrée durant ce test est que lorsque je faisais les tests avec des cartes sims avec des opérateurs français les sms étaient bien émis et reçus par le téléphone et le routeur. Mais lorsque j'utilisais la carte sim de la société, je pouvais communiquer du téléphone au routeur mais pas du routeur au téléphone. Après quelques échanges avec le fournisseur, j'ai supposé que le problème venait du « **Sms Center** ». Le « Sms center est ou alors SMSC (sigle pour Short Message Service Center ou centre de messagerie SMS) est un composant du réseau de téléphonie mobile permettant de gérer le transfert de messages SMS (textes ou binaires) entre téléphones mobiles. Dans notre cas, il devait être renseigné avec l'équipe qui s'occupe des cartes sims, j'ai donc contacté le service client **matooma**. Les cartes sims matooma ont pour principales avantages de fonctionnait dans tous les secteurs liés au domaine de l'Internet des Objets. En général elles sont déployées sont la santé, la sécurité, la télématique (géolocalisation) et la télémétrie (télérelève). Mais elle est parfaitement adaptée à notre projet.

J'ai donc obtenu les informations nécessaires à remplir voir **figure 41**

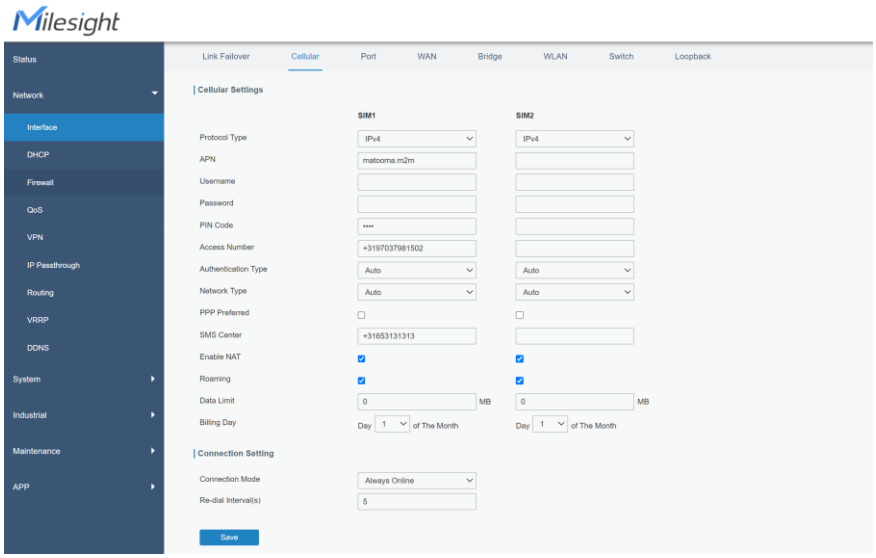
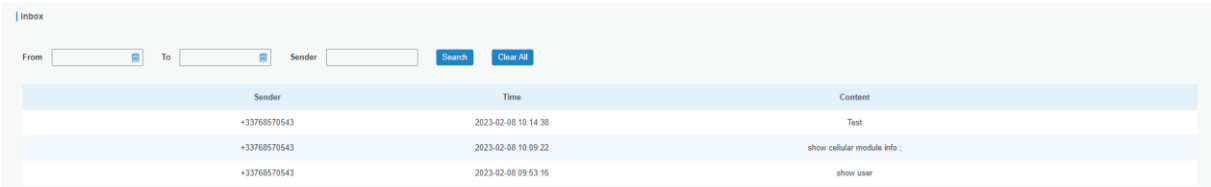


FIGURE 41 INFORMATIONS COMPLETEES DANS L'INTERFACE CELLULAR

Au niveau des résultats, j'ai pu par la suite émettre des sms de mon téléphone et émettre des sms du routeur. Dans les deux de figure les sms étaient bien reçus respectivement par le routeur et par le téléphone.

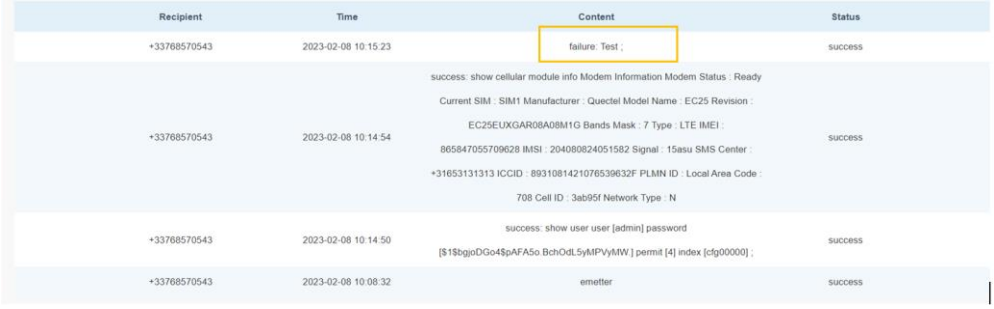


| Inbox

From To Sender Search Clear All

Sender	Time	Content
+33768570543	2023-02-08 10:14:38	Test
+33768570543	2023-02-08 10:09:22	show cellular module info ;
+33768570543	2023-02-08 09:53:16	show user

FIGURE 42 SMS ENVOYER D'UN TELEPHONE PORTABLE AU ROUTEUR RECEPTION OK



Recipient	Time	Content	Status
+33768570543	2023-02-08 10:15:23	failure: Test ;	success
+33768570543	2023-02-08 10:14:54	success: show cellular module info Modem Information Modem Status : Ready Current SIM : SIM1 Manufacturer : Quectel Model Name : EC25 Revision : EC25EUXGAR08A08M1G Bands Mask : 7 Type : LTE IMEI : 865847055709628 IMSI : 204080824051582 Signal : 15asu SMS Center : +31653131313 ICCID : 89310814210765396 32F PLMN ID : Local Area Code : 708 Cell ID : 3ab95f Network Type : N	success
+33768570543	2023-02-08 10:14:50	success: show user user [admin] password [1bgjoDGo4$pAFA5o.BchOdL5yMPVyMW.] permit [4] index [cfg00000] ;	success
+33768570543	2023-02-08 10:08:32	emetter	success

FIGURE 43 SMS ENVOYER DU ROUTEUR AU TELEPHONE PORTABLE RECEPTION OK

Sur la figure 43 on voit que « failure : test ; » a été émis du routeur au téléphone portable.

J'ai pu étudier certaine fonctionnalité en émettant certaine commande type comme « show cellular module info » ou encore « show user » on reçoit une réponse automatique du routeur sur les informations demandées par la commande. Autrement dit, la partie envoi de sms et demande de réponse automatique a été **validé**.

Le troisième test et pas des moindre

L'un des tests le plus important à réaliser est le test du Port RS485. Durant ce test, j'ai repris l'ancien code réalisé par Paul Fleury que j'ai modifié pour qu'il soit compatible avec le routeur. Ils s'agissaient de confirmer la réception des données sur le port RS485. Pour ça, il fallait d'abord rechercher le nom des ports/ interfaces physiques du port RS485 et du module GSM. Après quelques recherches avec le fournisseur du routeur, nous avons trouvé que l'interface qui correspond au modem cellular est « /dev/ttymxc1 ». Je devais préciser le port du module gsm pour permettre si besoin de pouvoir communiquer avec le routeur par sms en exécutant un code python. Et l'interface du port RS485 est « /dev/ttymxc2 ».

Par rapport à la structure du code initial, j'ai réalisé plusieurs modifications car le routeur a des capacités de mémoire vive moins important que le Rasberry Pi. Dans le projet python initial, plusieurs chemins sont spécifiés selon les dossiers de la Raspberry pi. J'ai modifié ces chemins selon les chemins trouvés sur le routeur. Pour cela, il a fallu connaitre les dossiers visibles dans le routeur. J'ai donc réalisé un code qui permet de se déplacer dans le routeur et affiche les fichiers et dossiers selon le chemin spécifié. Après avoir vérifié dans quel dossier se trouvait le projet implanté dans cette page. J'ai modifié la structure des fichiers du Projet « Datalogger » afin que tous les fichiers soient lus correctement.

Il fallait que le routeur soit capable de lire les fichiers de configuration voire **figure 46** (« fichiers de chaque centrale ») et les fichiers de définitions de chaque onduleur utilisés sur une centrale pour cela voici comment j'ai organisé la structure du dossier.

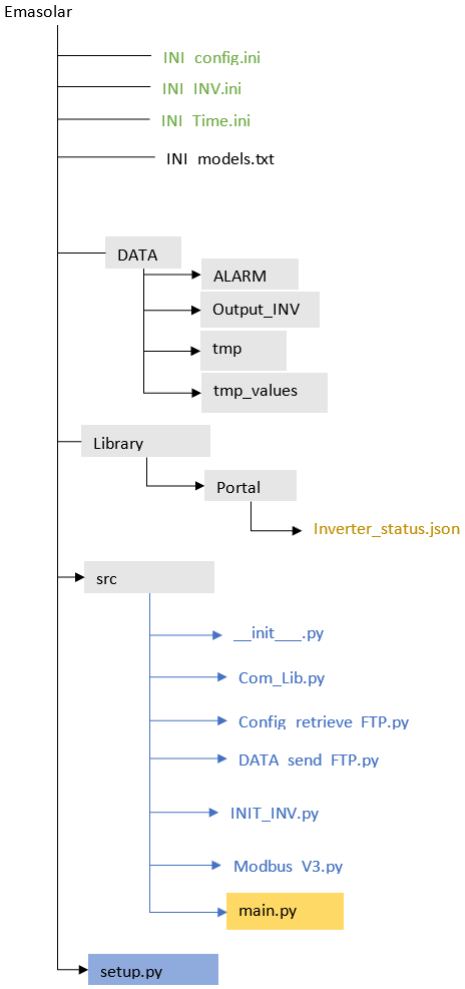


FIGURE 44 STRUCTURE DU PROJET DATALOGGER MIS DANS LE ROUTEUR

À partir de là je pouvais déjà observer que les fichiers étaient bien lus par le routeur. Cette étape m'a demandé beaucoup de patience et de la minutie car il a fallu faire une démarche empirique, sur ce que le routeur reconnaissait ou pas. Par la suite, j'ai debuggé le code selon le compilateur du routeur car la version de python utilisée dans le routeur est la version python2.7 J'ai donc réalisé plusieurs équivalences de code pour différentes fonctions.

- La fonction 'newline' permet créer un fichier csv dans la version python3, pour une version python2.7 il faut la remplacer par « r+b ».
- Idem pour la fonction 'astimezone' qui permet normalement de définir l'heure je suis donc passée par une équivalence la fonction 'datetime'. J'ai pu afficher l'horaire de

l'appareil en spécifiant que c'était l'horaire venant de Paris avec la fonction « pytz.timezone ».

Une fois que le code semblait capable de s'exécuter sans erreur avec mon tuteur nous avons mis en place une simulation de ce qui pouvait se passer sur une centrale. Au siège de la société à Lyon, nous avons placé trois panneaux solaires en série, nous l'avons relié à un onduleur SUNGROW. L'onduleur SUNGROW a été relié grâce à une liaison RS485 au routeur UR32 qui fait office de Datalogger. Et j'ai branché mon ordinateur au routeur pour pouvoir lancer le code et voir le déroulement du code.

Voici un schéma qui montre l'installation :

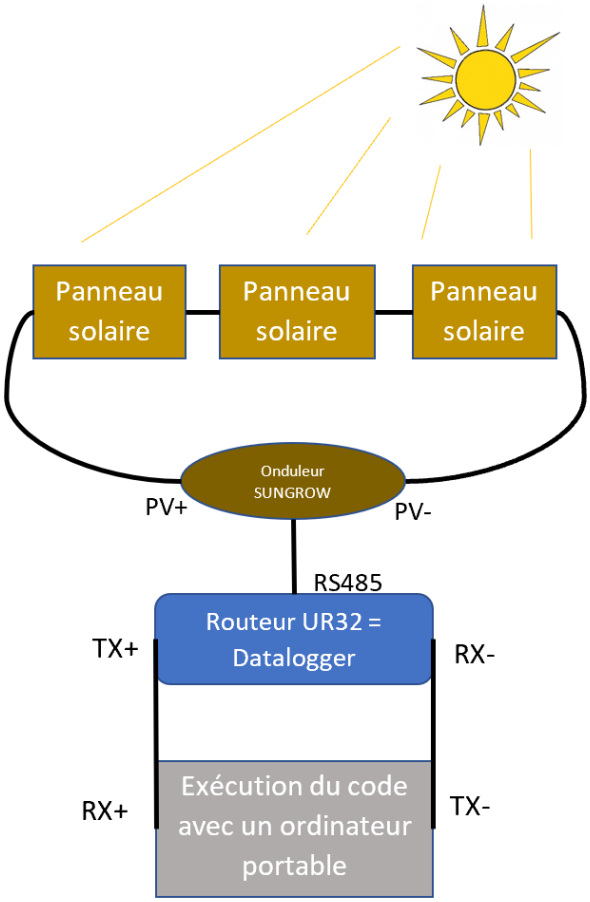


FIGURE 45 SIMULATION D'UNE INSTALLATION PHOTOVOLTAÏQUE

Les résultats

Durant ce test, j'ai pu voir que les fichiers csv se créaient bien toutes les 10 min. À l'intérieur du fichier csv, les caractéristiques demandées à l'onduleur étaient bien copiées dans chaque fichier, cependant les valeurs ne se remplissaient pas comme attendues. J'ai

supposé que cela était due au faites des réglages du fichier de définition. Voici ce que je pouvais obtenir à l'affichage.

```
root@ROUTER:/usr# cd ..
root@ROUTER:/# python /usr/pyuser/app/emasolar/src/main.py
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##################################################################
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##             INITIALISATION DES VARIABLES ONDULEURS            ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##################################################################
fonction d'affichage des infos importantes
07/02/23 09:32:51 ## File: INI_Time.ini found! ** OK                                ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ## File: INI_INV.ini found! ** OK                                 ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ## File: INI_config.ini found! ** OK                              ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ## File: SUNGROW/SUNGROW_Modbus_v3.txt found! ** OK               ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ## File: stat_code.txt found! ** OK                               ##
ourir un fichier csv
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##################################################################
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##                      INITIALISATION TERMINEE                  ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##################################################################
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##                                 MODBUS                        ##
fonction d'affichage des infos importantes
07/02/23 09:32:51 ##################################################################
fonction d'affichage des infos importantes
07/02/23 09:32:51 ## Communication en protocole ModBus                              ##
Fonction principal dans la mise en place du csv et lancement des requête
trouver les fichiers d'alarme
Verification de la creation d'un fchier json
Calcul d'une date/heure 10 minutes plus tard
```

FIGURE 46 EXECUTION DU PROJET DATALOGGER

On peut voir sur la **figure 46** que lorsque je lance l'exécution du code « main.py » les fichiers de configuration sont trouvés et le fichier de définition SUNGROW est bien trouvé, une fois l'initialisation terminée la communication Modbus est lancée. Et Le code doit normalement récolter les valeurs demandées.

Les fichiers csv qui remplissent le tableau de valeur sont générés toutes les 10min et toutes les 59 min le fichier est compressé « en .zip » et envoyé sur le serveur FTP. Ici l'exécution du code s'est arrêtée au moment de transformer en fichier « .zip ».

```
BusyBox v1.25.1 (2021-08-27 21:01:50 CST) built-in shell (ash)

root@ROUTER:~# ls
root@ROUTER:~# cd  /usr/pyuser/app/emasolar/DATA/Output_INV
root@ROUTER:/mnt/mmcblk0p1/pyuser/app/emasolar/DATA/Output_INV# ls
_2022_11_17_09_00.csv  _2023_02_07_09_00.csv  _2023_02_07_10_00.csv  _2023_02_07_11_00.csv  _2023_02_07_12_00.csv  _2023_02_07_13_00.csv
root@ROUTER:/mnt/mmcblk0p1/pyuser/app/emasolar/DATA/Output_INV#
```

FIGURE 47 LES FICHIERS CSV DU PROJET

À ce stade de mon avancée, nous arrivons à la fin de ma période de stage, les résultats sont satisfaisants et seul la mise en place d'un code python plus performant et plus adaptée doit être mis en place.

utbm
université de technologie
Belfort-Montbéliard

Comparaison entre les routeurs testés :

	Routeur 4G LTE WLINK R220	Routeur 4G Wifi industriel - UR32
Prix	268 euros HT	201 euros HT
Structure	La structure est comme l'utilisateur le veut	La structure est imposée par le fabricant
Ports	Port WAN non compris	Port WAN compris
Mémoire	Pas de carte SD	Carte SD
Résultats	Le code n'a pas pu être exécuter sans les onduleurs seulement la fonction d'interpréteur python	Le code à pu être exécuter avec la mise en place des onduleurs

6) Améliorations

- Pour un projet qui se situe dans le secteur de l'électronique et système embarqué, j'estime que des améliorations sont toujours possibles notamment dans le processus d'envoi des fichiers à distance. En effet, mon tuteur avait pensé à l'envoi des fichiers grâce à un VPN. Nous avions choisi Open VPN car c'est un logiciel libre qui permet de créer un réseau privé virtuel est qui est accessible à mettre en place. J'avais commencé à mettre en place le VPN mais j'ai préféré me concentrer sur le bon fonctionnement du programme principal.
- Une autre amélioration possible est l'envoi des sms automatique par le routeur via un code python. Jusqu'à présent, j'ai pu mettre en place l'envoi des sms automatique du routeur uniquement via l'interface Milesight. L'objectif est que lorsque le code python s'exécute selon l'envoi d'un sms d'un téléphone portable au routeur, le routeur répond « bien reçu » par exemple.
- Bien évidemment, une amélioration principale est celle réalisée durant le troisième test, la confirmation en un fichier « .csv » en fichier « .zip ». Plus globalement, il faudrait terminer le débogage pour confirmer le bon fonctionnement du projet « Datalogger ».

7) Mon expérience de stagiaire tout au long du semestre

8) Conclusion

Pour terminer, ce stage que j'ai réalisé dans la ville de Lyon au sein de la société Emasolar, m'a beaucoup apporté que ça soit dans le domaine technique, mais également d'un point de vue personnel. En tant que future ingénieure en énergie, il a été une étape essentielle dans la prise de décision de m'orienter vers une filière de la production d'énergie électrique. C'est également une fierté d'avoir trouvé un projet aussi stimulant et d'y avoir apporter ma contribution. Je repars convaincu que ce projet aboutira. J'ai pu détailler dans les paragraphes précédents mon ressenti.

Je remercie donc toutes les personnes avec lesquels j'ai pu travailler et qui m'ont aidé pour le projet « Datalogger.

Bibliographie

Livre de GERARD, 2012, Swinnen Apprendre à programmer avec Python 3

Lionel Cassin, 2023 https://www.emasolar.com/

Olga Weis, 2021 https://www.eltima.com/fr/article/rs485-communication-guide/

Embedded Micro Technology, 2023, https://www.embeddedpi.com/integrator-board

Paul fleury, 2021, rapport de stage

WLINK, 2022, WL-R220 OpenWrt Router Quickstart V2.4@201906__6758652231

WLINK, 2022, WL-R220(MTK) OpenWRT Quickstart Guide v1.0_20220325

WLINK, 2022, WL-R220(MTK) OpenWrt Router Quickstart_202210_V1.1

Milesight, 2022, ur32-datasheet-en

Milesight, 2022, ur32-quick-guide-en

Milesight, 2022, ur32-user-guide-en

JDN, 2023, https://www.journaldunet.fr/web-tech/dictionnaire-du-webmastering/1445298-rom-memoire-informatique-definition-concrete-et-caracteristiques-pour-le-mobile/

Matooma, 2023, https://www.matooma.com/

Openwrt, 2023, https://openwrt.org/

Sparwan, 2023, https://sparwan.com/

Wikipedia, 2023, https://fr.wikipedia.org/wiki/Short_Message_Service_Center

Milesight, 2022, https://support.milesight-iot.com/s

Annexes

Annexe 1 :

Modbus RS485 est un protocole de communication développé pour les réseaux industriels en environnements difficiles tels que les usines, les centrales électriques, les systèmes de contrôle de processus, etc. Il utilise une liaison série RS485 pour transmettre les données, ce qui permet de couvrir de grandes distances et de connecter jusqu'à 32 dispositifs sur un seul réseau.

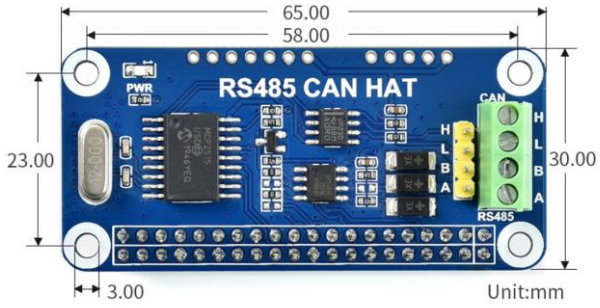
Le protocole Modbus RS485 est basé sur un système maître/esclave, où un dispositif maître envoie des requêtes aux dispositifs esclaves et les esclaves répondent avec les données demandées. Le dispositif maître peut être un ordinateur ou un automatisme qui surveille et contrôle les dispositifs esclaves. Les dispositifs esclaves sont généralement des actionneurs, des capteurs, des entrées/sorties, etc.

Modbus RS485 utilise un format de requête/réponse pour échanger des données entre les dispositifs. Les requêtes contiennent des informations sur la fonction demandée et l'adresse du dispositif esclave, tandis que les réponses contiennent les données demandées. Le protocole prend en charge de nombreuses fonctions, telles que la lecture et l'écriture de données enregistrées, la surveillance de l'état des entrées et sorties, etc.

Modbus RS485 est un protocole ouvert et facile à utiliser, ce qui en fait un choix populaire pour les applications industrielles. Il est également compatible avec de nombreux systèmes d'exploitation différents, ce qui le rend facile à intégrer dans des systèmes existants.

Annexe 2 :

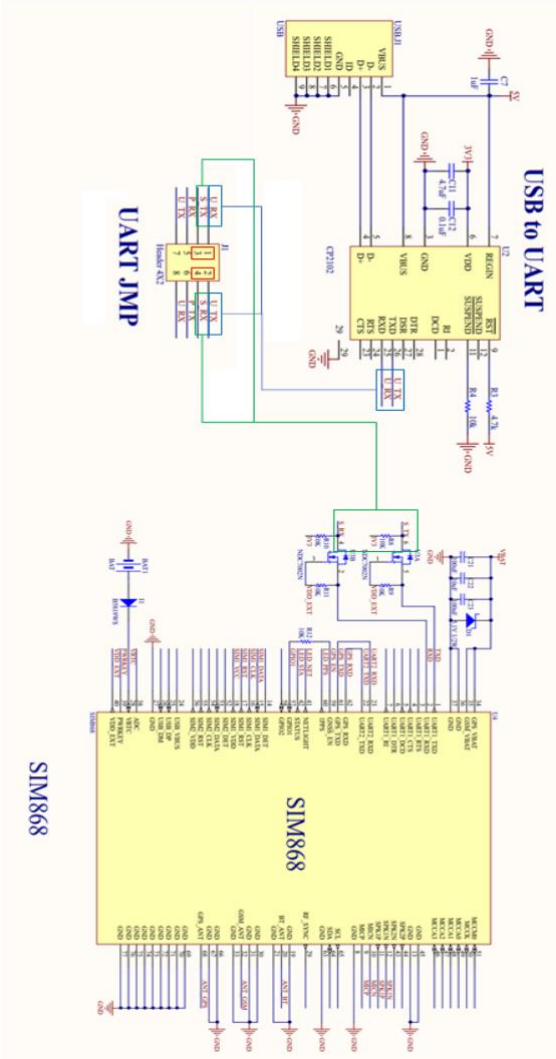
Dimensions



RS485 CAN HAT FOR RASPBERRY PI - TECHNICAL SPECIFICATIONS:

• **Operating Voltage**	– 3.3V DC
• **CAN Controller**	– Microchip MCP2515
• **CAN Transceiver**	– Texas Instruments SN65HVD230
• **RS-485 Transceiver**	– Sipex SP3481/SP3485
• **Circuit Protection**	– Onboard TVS (Transient Voltage Suppressor)
• **Raspberry Pi Compatibility**	– Raspberry Pi 4B | 3B+ | 3B | 2B – Raspberry Pi Zero W | Zero | Zero WH – Raspberry Pi Boards with 40 GPIO Headers
• **Dimensions**	– 65 x 30mm

Annexe 3 : Schéma électronique et jumpers du module Sim



Annexe 4 : Les mémoires

NOR :

- La mémoire Flash NOR offre une plus grande vitesse, car elle possède des capacités d'accès aléatoire, un bon choix pour les appareils mobiles, car elle est adaptée pour stocker de grandes quantités de code.
- La fonction d'effacement et d'écriture est plus lente que la NAND.
- Elle accueille une taille de cellule de mémoire plus grande que la NAND.
- La limitation des capacités de mise à l'échelle rend la densité de bits possible.

NAND :

- Le système de type NAND fournit des capacités de réécriture et d'effacement rapides, mais est un peu plus lente que la NOR en vitesse de lecture.
- La mémoire Flash NAND a une excellente fiabilité et une très bonne tenue dans le temps. Elle est parfaite pour la vidéo numérique, le stockage de données sur carte ou SSD et la musique. Cela est dû à la haute vitesse et à la haute capacité de NAND, ainsi qu'à une mémoire Flash qui est bon marché, car le coût par bit est généralement plus faible.
- Réécriture rapide des données.
- Stocke une grande quantité de données et est couramment utilisée pour les clés USB, les smartphones et caméras.

Annexe 5 : Les bibliothèques pythons à installer dans le routeur WLINK R220

```
Installing python3 (3.6.12-1) to root...
Collected errors:
 * satisfy_dependencies_for: Cannot satisfy the following depe
 *      python3-light
 *      python3-asyncio
 *      python3-cgi
 *      python3-cgitb
 *      python3-codecs
 *      python3-ctypes
 *      python3-dbm
 *      python3-decimal
 *      python3-distutils
 *      python3-email
 *      python3-gdbm
 *      python3-logging
 *      python3-lzma
 *      python3-multiprocessing
 *      python3-ncurses
 *      python3-openssl
 *      python3-pydoc
 *      python3-sqlite3
 *      python3-unittest
 *      python3-xml
 * opkg_install_cmd: Cannot install package python3.
root@OpenWrt:/tmp#
```

Annexe 6 : La mémoire dans le routeur WLINK R220

```
PS C:\Users\Monia\Desktop\test\routeur\git> scp -r RPI root@192.168.1.1:
root@192.168.1.1's password:
.gitignore                                                    100%  354    42.4KB/s   00:00
Com_Lib.py                                                    100%   14KB 139.7KB/s   00:00
HUAWEI_Modbus_v0.txt                                          100% 3465   138.4KB/s   00:00
HUAWEI_Modbus_v2.txt                                          100% 8599   135.1KB/s   00:00
HUAWEI_Modbus_v3.txt                                          100% 4841   134.3KB/s   00:00
HUAWEI_Modbus_v4.txt                                          100% 2757   128.7KB/s   00:00
stat_code.txt                                                 100% 2826   132.1KB/s   00:00
INI_config.ini                                                100%  312    37.4KB/s   00:00
INI_INV.ini                                                   100%  289    37.0KB/s   00:00
INI_Time.ini                                                  100%  209    26.8KB/s   00:00
INV_models.txt                                                100%  548    61.7KB/s   00:00
KACO_Modbus_v1.txt                                            100% 2405   124.7KB/s   00:00
KACO_sunspec.txt                                              100%  242    33.1KB/s   00:00
KACO_var_sunspec.txt                                          100% 1872   115.9KB/s   00:00
KOSTAL_Modbus_v1.txt                                          100% 8711   137.0KB/s   00:00
KOSTAL_sunspec.txt                                            100%  527    56.1KB/s   00:00
KOSTAL_var_sunspec.txt                                        100% 6798   140.1KB/s   00:00
maker.sh                                                      100% 2033   108.2KB/s   00:00
requirements.txt                                              100% 1673   106.9KB/s   00:00
__init__.py                                                   100%    0     0.0KB/s   00:00
__init__.cpython-310.pyc                                      100%  151    20.7KB/s   00:00
__init__.cpython-37.pyc                                       100%  132    19.5KB/s   00:00
Config_retrieve_FTP.py                                        100%   14KB 108.5KB/s   00:00
LATEST_ALARM.txt                                              100%  153    20.9KB/s   00:00
ALARM_210401_145058.csv                                       100%  285    32.1KB/s   00:00
ALARM_210401_145457.csv                                       100%  285    34.2KB/s   00:00
ALARM_210401_145948.csv                                       100%  293    35.1KB/s   00:00
ALARM_210401_150024.csv                                       100%  285    14.0KB/s   00:00
ALARM_210401_150617.csv                                       100%  293     7.2KB/s   00:00
ALARM_210401_150709.csv                                       100%  285     7.9KB/s   00:00
ALARM_210401_151416.csv                                       100%  293     7.8KB/s   00:00
ALARM_210401_151508.csv                                       100%  285     7.5KB/s   00:00
ALARM_210401_151946.csv                                       100%  293     8.0KB/s   00:00
```

Annexe 7 : Le fonctionnement de la carte sim matooma

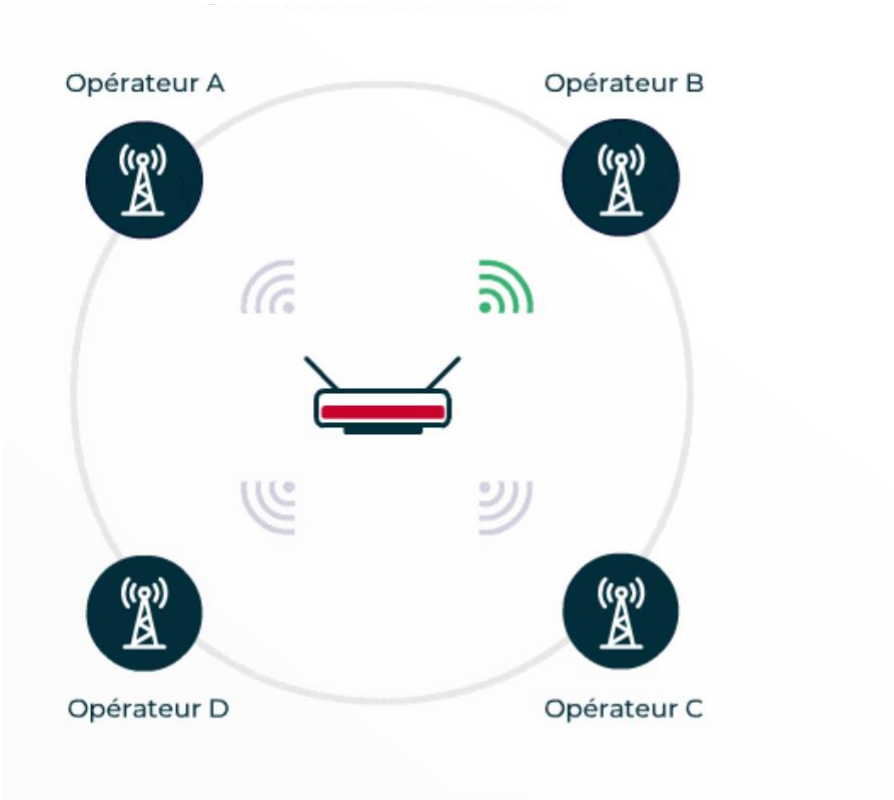


FIGURE 48 FONCTIONNEMENT DE LA CARTE SIM MATOOMA

Mots clefs

10 Matériels électriques, électroniques, informatiques – 12 Energie – 03 Etude/Développement – 08 Maintenance – 11 Développements logiciels – 33 Prototypages –01 Documentation – 05 Logiciels de réseaux

INZOUDDINE Monia

Rapport de stage ST40 - A2022

Résumé

De nos jours, on assiste à une forte croissance de l'utilisation de panneaux photovoltaïques par les entreprises ou encore par les particuliers. Afin de garantir une efficacité de production des panneaux photovoltaïques leur maintenance est nécessaire. Le suivi à distance de l'efficacité des panneaux solaires se fait grâce au monitoring. Le monitoring permet l'affichage de certaines données à travers des courbes et des données. Pour collecter ces données l'appareil utilisé est un datalogger. Ainsi le but du stage est de réaliser des tests avec de nouveaux appareils qui vont servir de datalogger. Pour cela, il y a une partie électronique pour adapter l'appareil, une partie informatique pour collecter les données et les traiter et la partie test.

Les différentes étapes de gestion de projet sont présentées ainsi que des pistes d'améliorations.

Après une analyse approfondie de l'équipement et des projets existants nous avons convenu que ce qui convenait le mieux est un routeur avec un port RS485, carte Sim, et un port Lan.

Le but est d'obtenir une communication entre l'onduleur via le RS485 et envoyer les données sur un serveur FTP grâce aux GPRS grâce à une carte SIM.

Entreprise Emasolar
25 rue Saint Jean de Dieu, Bâtiment D
69 007 Lyon
www.emasolar.com

utbm
université de technologie
Belfort-Montbéliard